Processingをはじめよう

第8章

動き

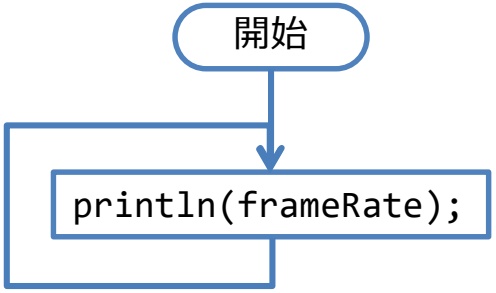
目次

- フレーム
- スピードと方向
- 2点間の移動
- 乱数
- タイマー
- 円運動
- Robot 6: Motion

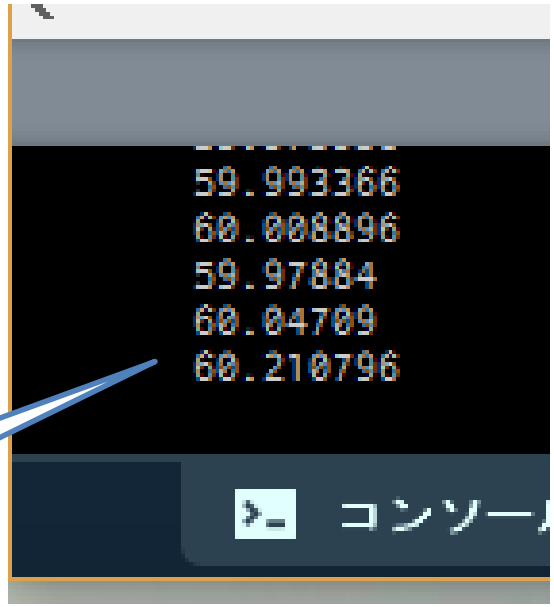
フレーム

- Example 8-1 フレームレートを見る

```
void draw() {
  println(frameRate);
}
```



- frameRate変数
  - 教科書 P225
  - 1秒間に何回画面が更新するか
  - 通常 1秒で60フレーム



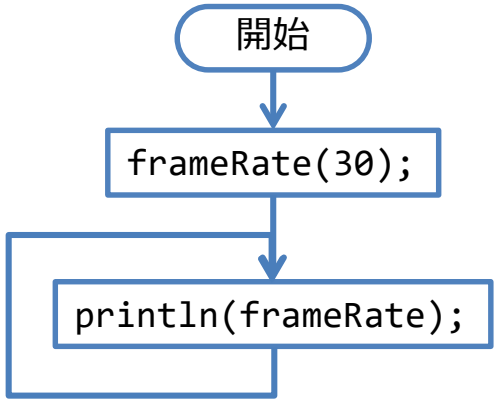
コンソールに
フレームレート表示

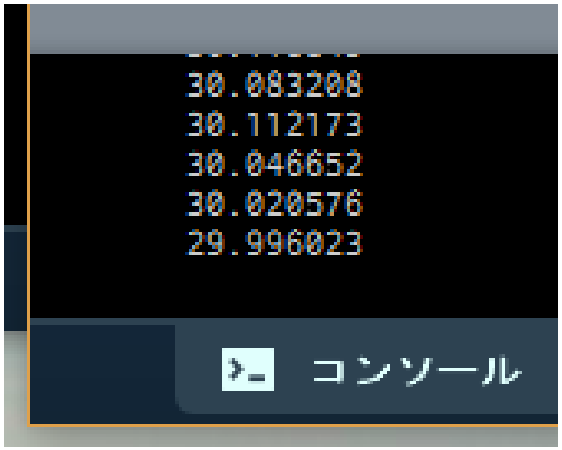
フレーム

- Example 8-2 フレームレートを見る

```
void setup() {
  frameRate(30);          //毎秒30フレームに設定
  //frameRate(12);        //毎秒12フレームに設定
  //frameRate(2);         //毎秒2フレームに設定
  //frameRate(0.5);       //毎秒0.5フレームに設定
}

void draw() {
  println(frameRate);
}
```

設定を変更して
フレームレートを確認





- frameRate()関数
  - 教科書 P225
  - フレームレートを設定

スピードと方向

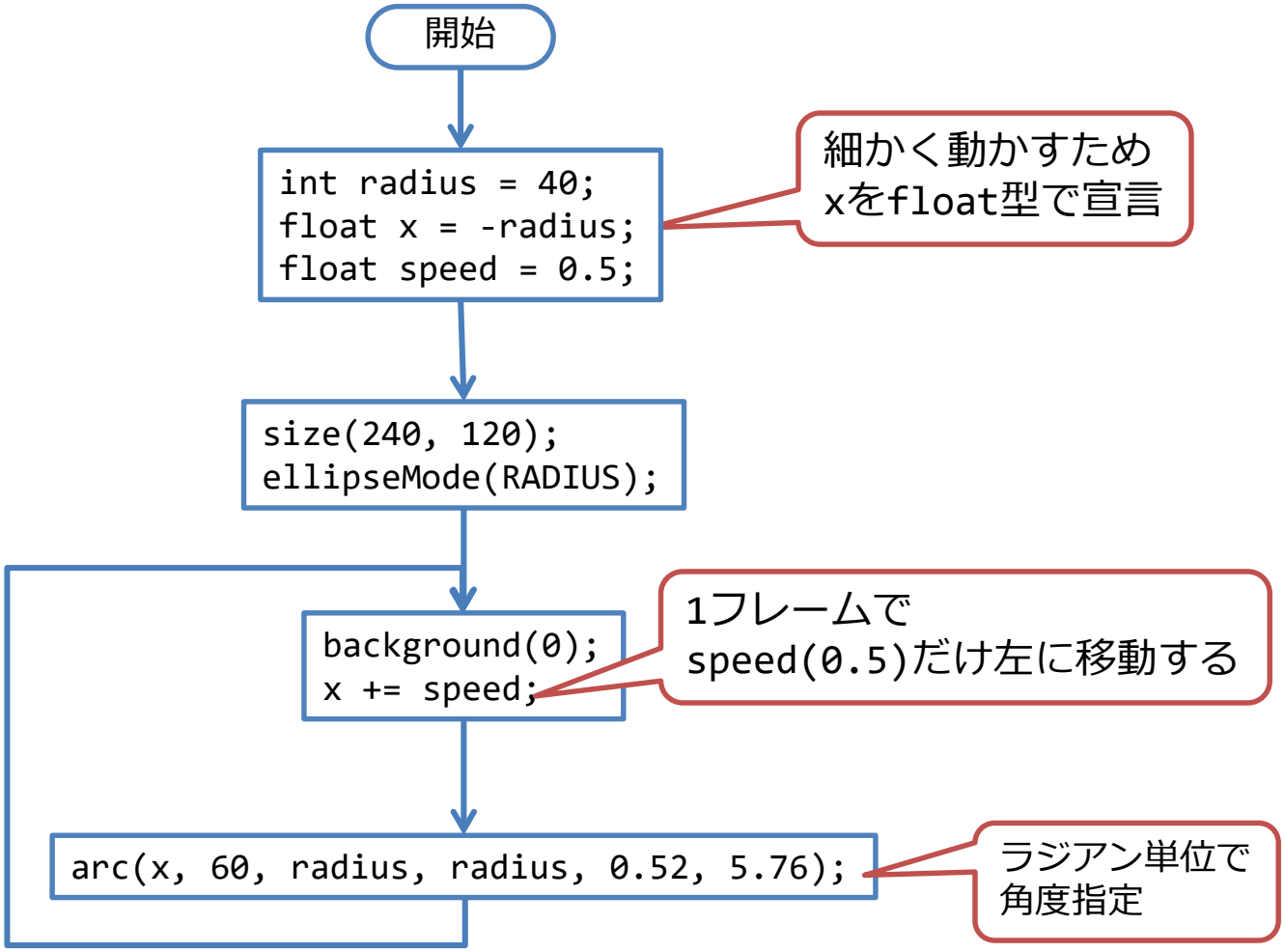
- Example 8-3 図形の移動

```
int radius = 40;          // 図形の半径radius       初期値40
float x = -radius;        // 図形のx座標            初期値 -radius
float speed = 0.5;        // 図形の移動する値speed  初期値0.5

void setup() {
  size(240, 120);
  ellipseMode(RADIUS);    // 円を半径を使って指定する
}

void draw() {
  background(0);
  x += speed;             // xをspeed加算する(xを左にspeedだけ移動)
  arc(x, 60, radius, radius, 0.52, 5.76); //円弧を描く
}
```

- 角度を指定して円弧を書く
  - arc(x, y, w, h, start, stop)　start：開始角度, stop：終了角度

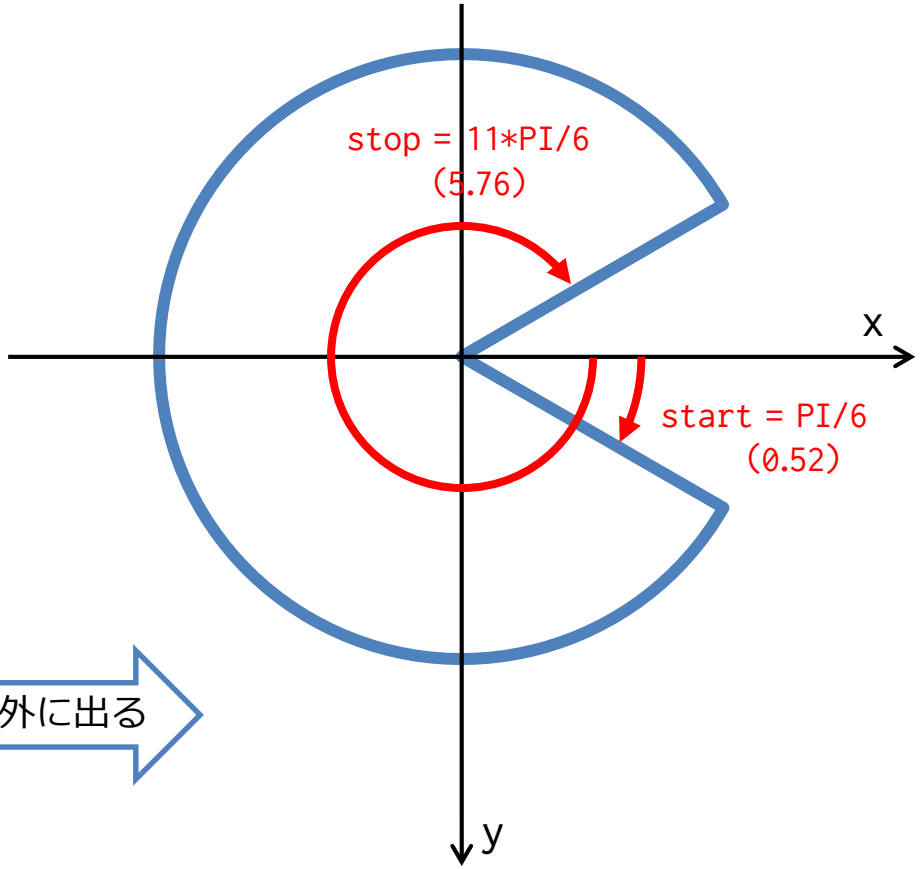
スピードと方向

- フローチャート



実行画面　開始・終了角度

開始角度 start　30度（PI/6）
終了角度 stop　330度（11*PI/6）



スピードと方向

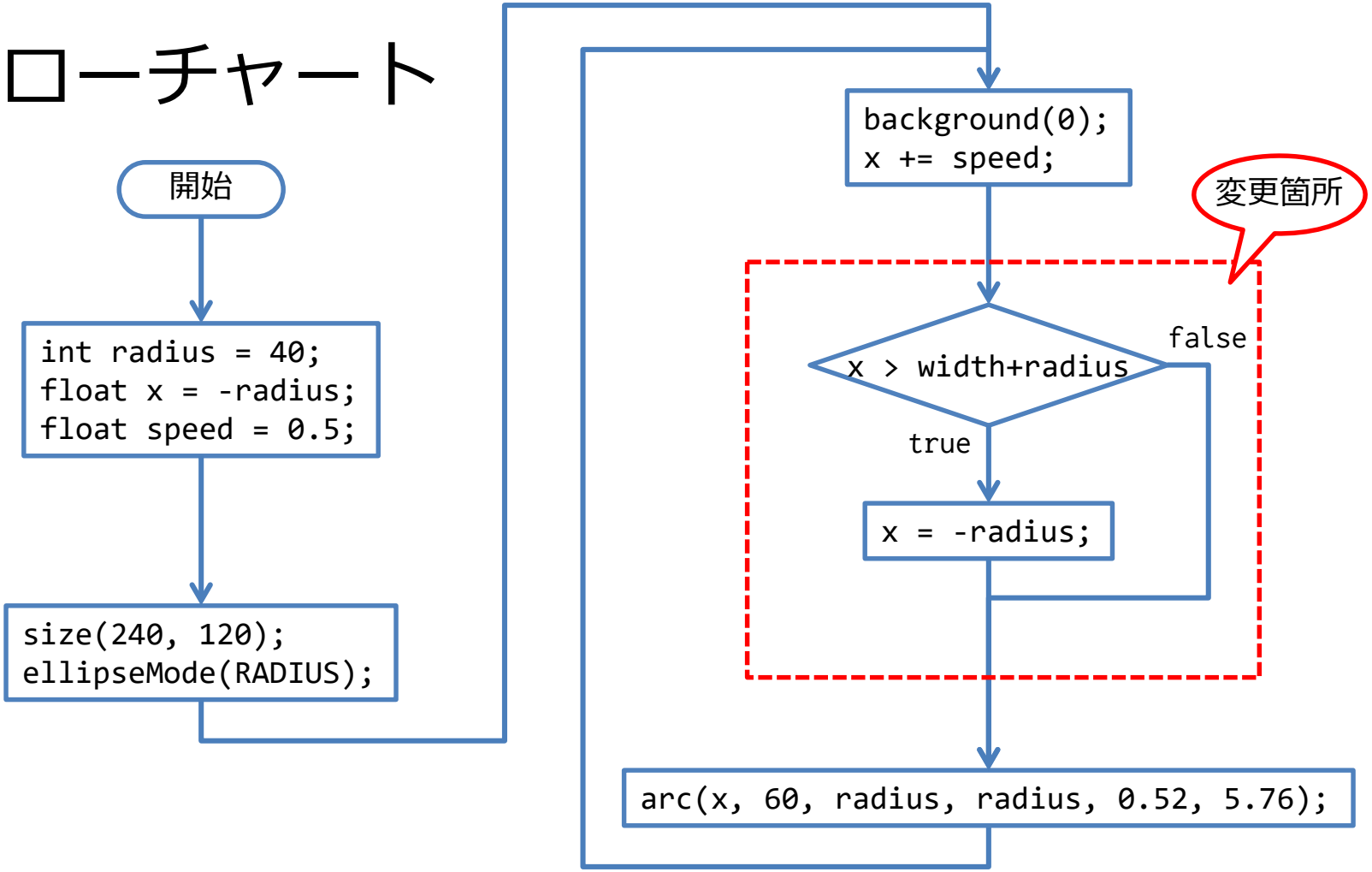
- Example 8-4 円筒形を回す

```
int radius = 40;
float x = -radius;
float speed = 0.5;

void setup() {
  size(240, 120);
  ellipseMode(RADIUS);
}

void draw() {
  background(0);
  x += speed;                    // xをspeedだけ加算
  if (x > width+radius) {        // 右の画面の外に出たら
    x = -radius;                 // 左から戻ってくる
  }
  arc(x, 60, radius, radius, 0.52, 5.76);
}
```

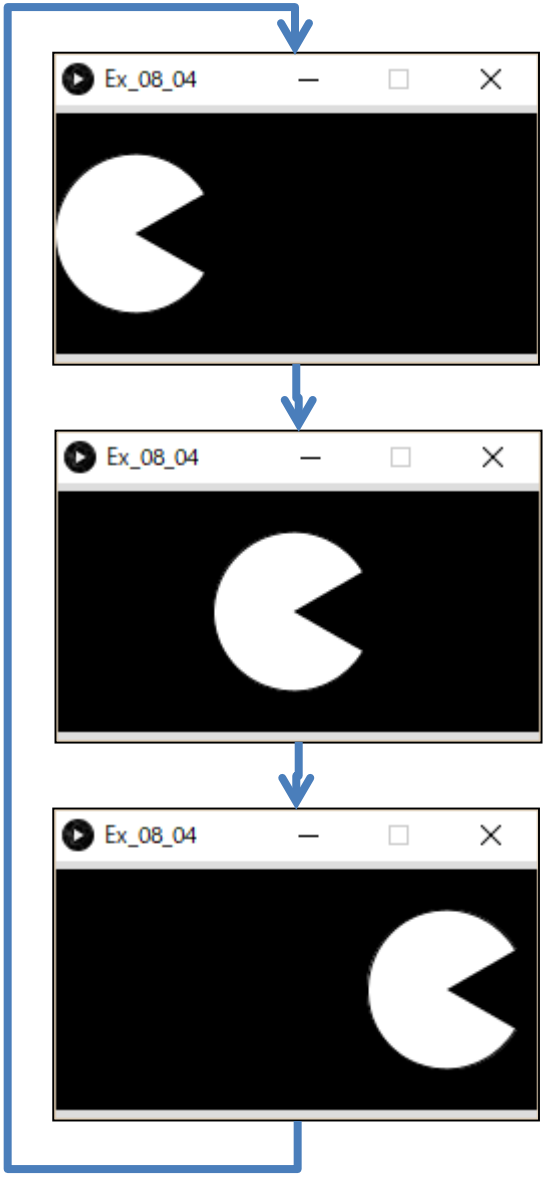
変更箇所

スピードと方向

- フローチャート



実行画面



- 図形の初期位置は
  左端外側（x = -radius）
- 図形が左から右へ移動
  （x += speed）
- 右側から外へ出たら
  左側から入ってくる

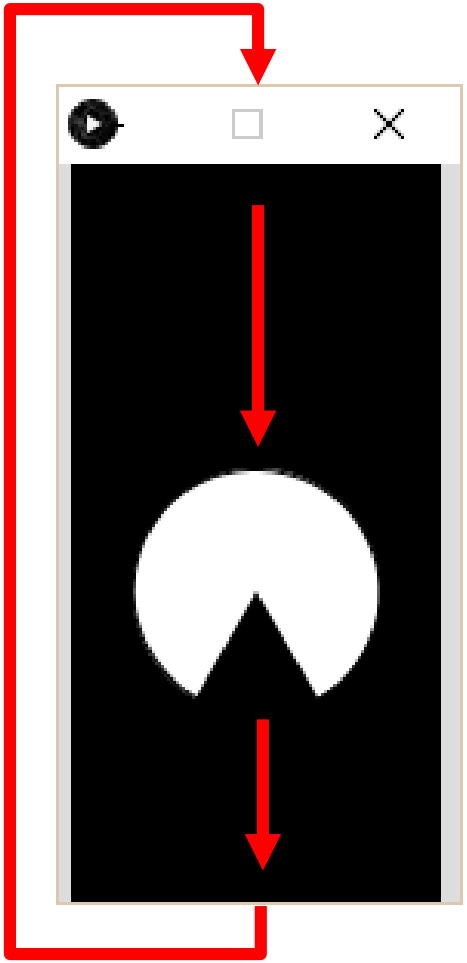
```
if(x > width+radius){
  x = -radius;
}
```

Kadai_08_01

- Example8-4を参考に、円弧を上から下に動かすプログラムを作る
  - ウィンドウサイズを(120, 240)
  - 変数xからyに変更
  - 下から外へ出たら、上から入ってくるように動かすこと
  - 円弧の角度は講義資料の最後のページ下向きの場合の角度を参考にする

実行結果例

- ウィンドウサイズを(120, 240)に変更
- 変数yを追加する
- 円弧が下から実行ウィンドウ外へ出たら、上から出てくるように動かすこと
- 円弧の角度は講義資料の最後のページ下向きの場合の角度を参考にする

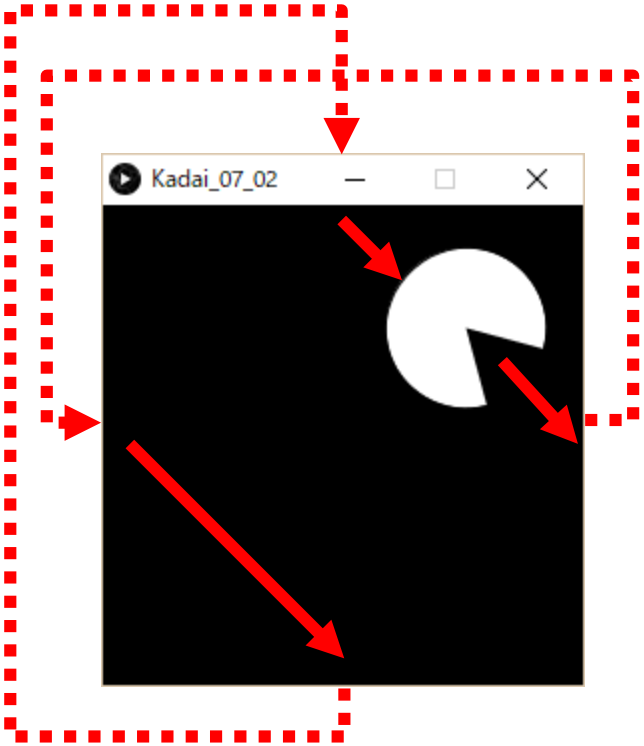


Kadai_08_02

- Example 8-4を参考に、円弧を右下45度方向に動かすプログラムを作る
  - ウィンドウサイズを(240, 240)
  - 変数xとyを宣言、初期値はx = 120とy = 0
  - 下端から外へ出たら上端から出てくる、右端から外へ出たら左端から出てくる
  - 円弧の開始角度はPI/4+PI/6(45度+30度), 終了角度はPI/4+11*PI/6(45度+330度)で指定

実行結果例

- ウィンドウサイズを
  (240, 240)に変更
- 変数xとyを宣言、
  初期値はx = 120, y = 0
- 下から外へ出たら上から出てくる
- 右から外へ出たら左から出てくる
- 円弧の開始角度は
  PI/4+PI/6(45度+30度),
  終了角度は
  PI/4+11*PI/6(45度+330度)で
  指定



スピードと方向

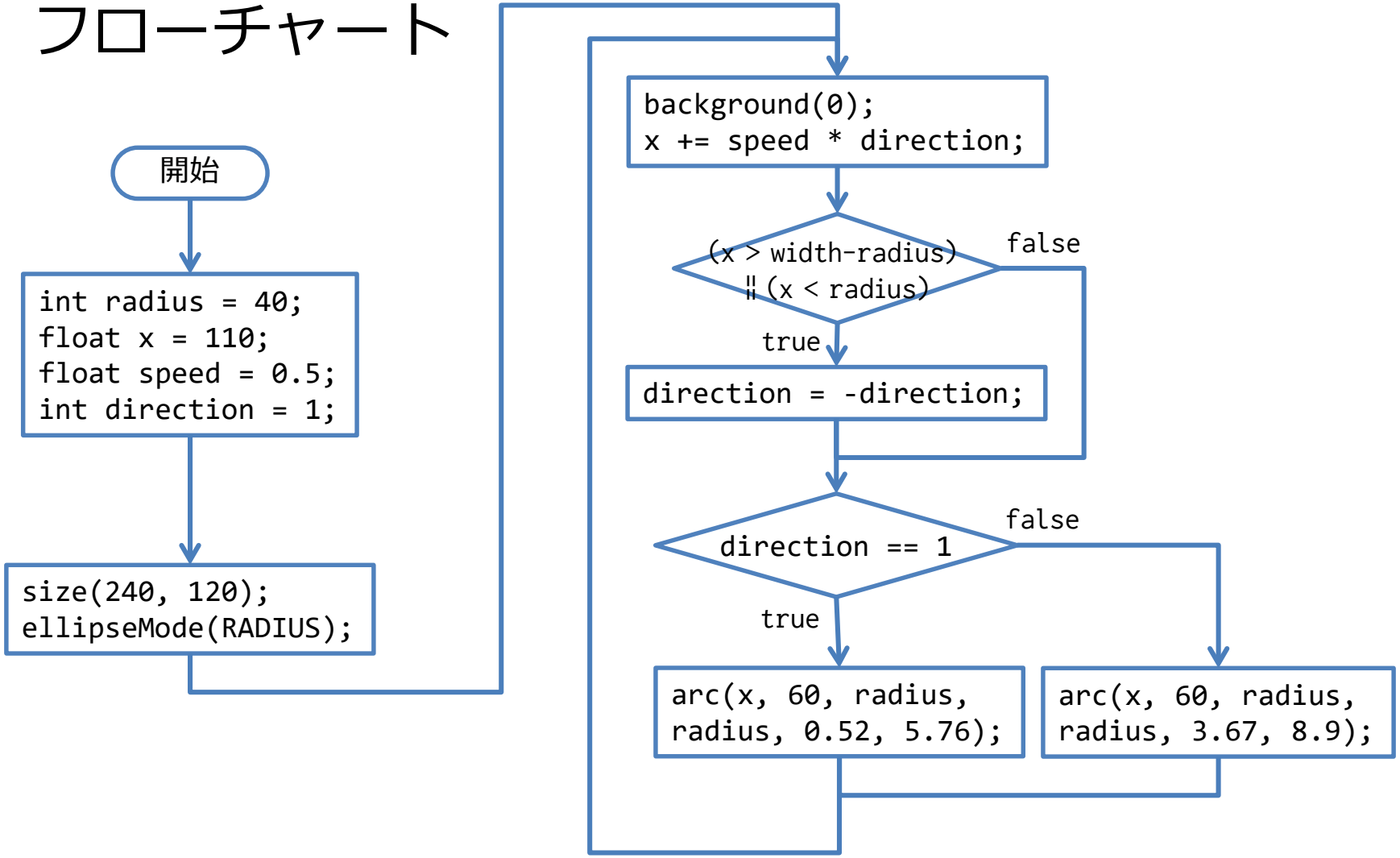
- Example 8-5 壁に当たって跳ね返る

```
int radius = 40;   // 図形の半径
float x = 110;     // x座標の初期値110
float speed = 0.5; // 1フレームで移動する量
int direction = 1; // 移動する方向(1:右、-1:←)

void setup() {
  size(240, 120);
  ellipseMode(RADIUS);
}
```

```
void draw() {
  background(0);
  x += speed * direction; //(移動量)×(方向)加算
  if ((x > width-radius) || (x < radius)) { //衝突したら
    direction = -direction; // 方向を反転
  }
  if (direction == 1) { // 進行方向が右の場合
    arc(x, 60, radius, radius, 0.52, 5.76); //口が右向き
  } else { //それ以外
    arc(x, 60, radius, radius, 3.67, 8.9);  //口が左向き
  }
}
```

スピードと方向

- フローチャート



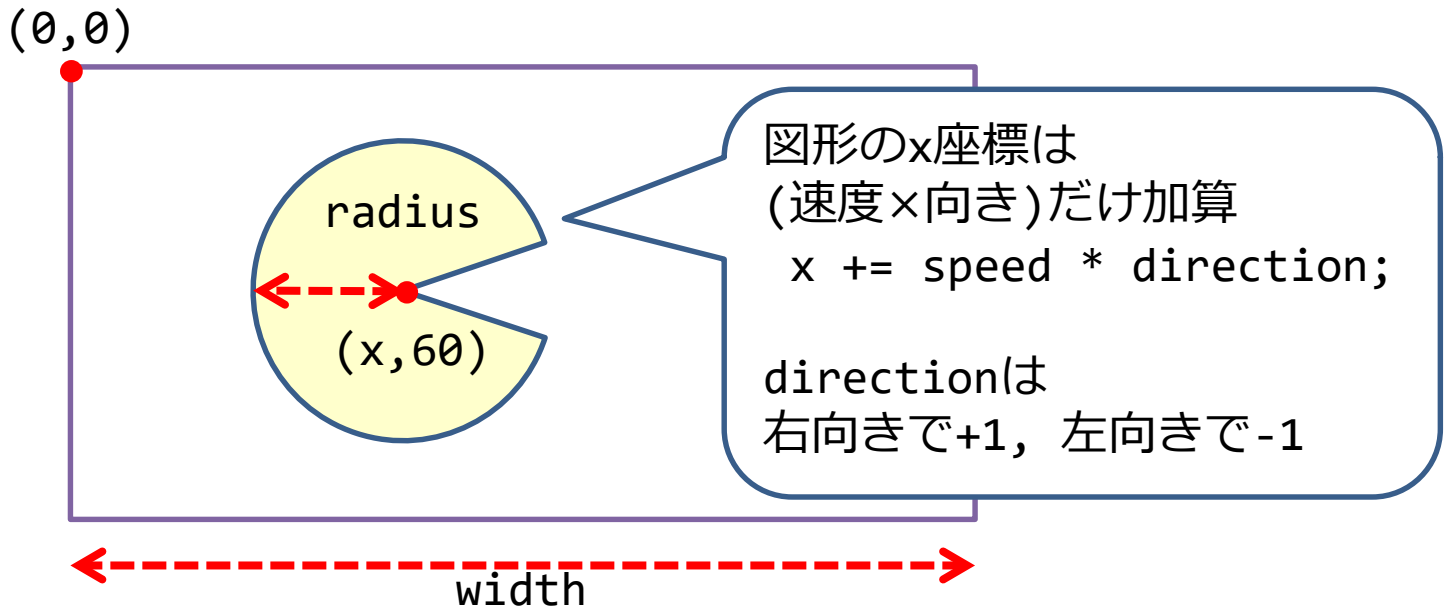
スピードと方向

- ポイント
  - 左端・右端で図形を反射させる
  - 移動量を(speed * direction)に変更
    - directionの符号を変えると移動方向を変更できる
  - 反射する条件：2つの条件のどちらかが真の場合
    - 右端： x > width-radius
    - 左端： x < radius
  - 条件が真のとき、移動方向を反転する
    - direction = -direction;

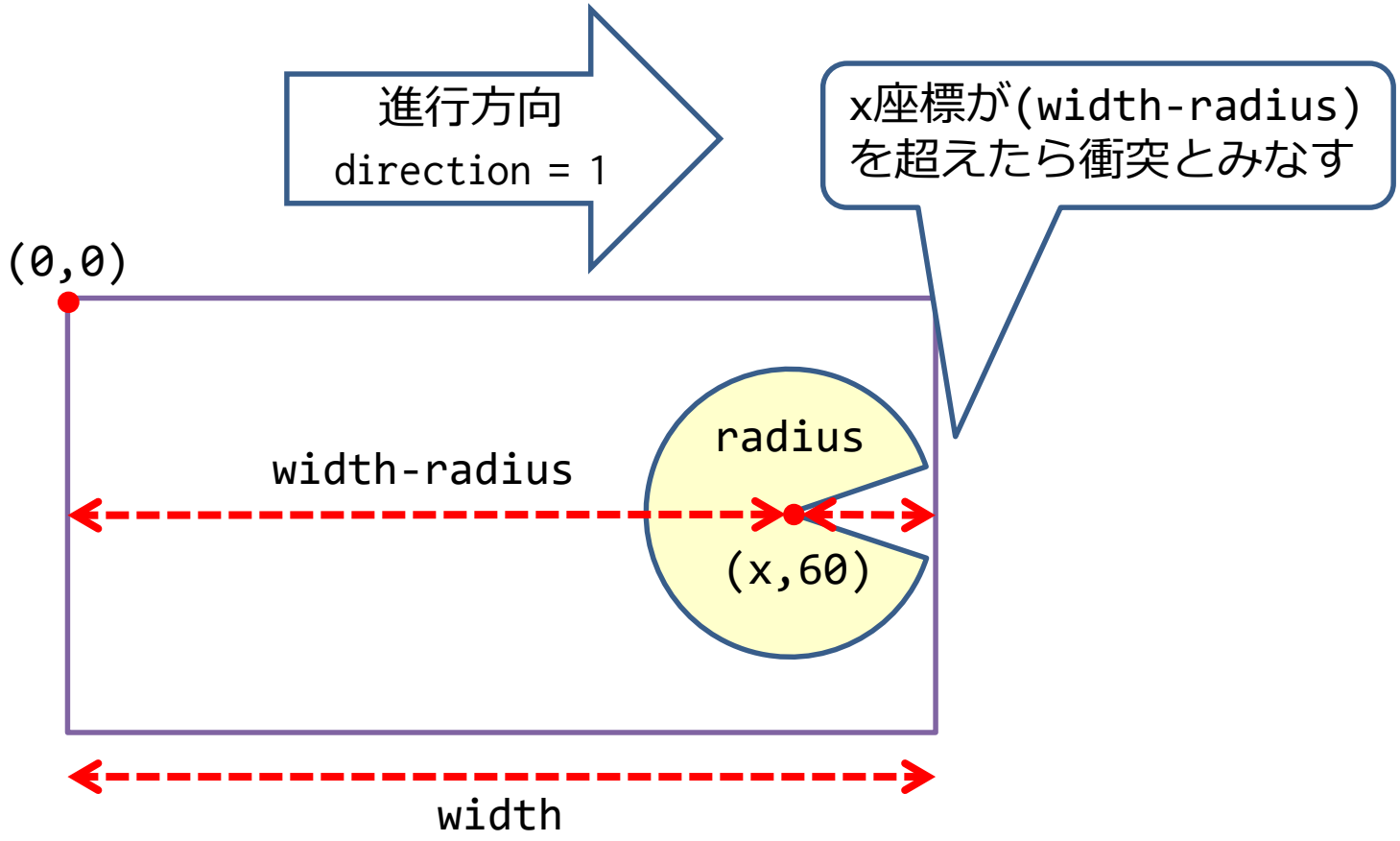
符号を変えて方向を反転

実行結果

Example_7_5
Example_7_5
Example_7_5
右へ移動して…
右端で反射
左へ移動して…
左端で反射…
を繰り返す

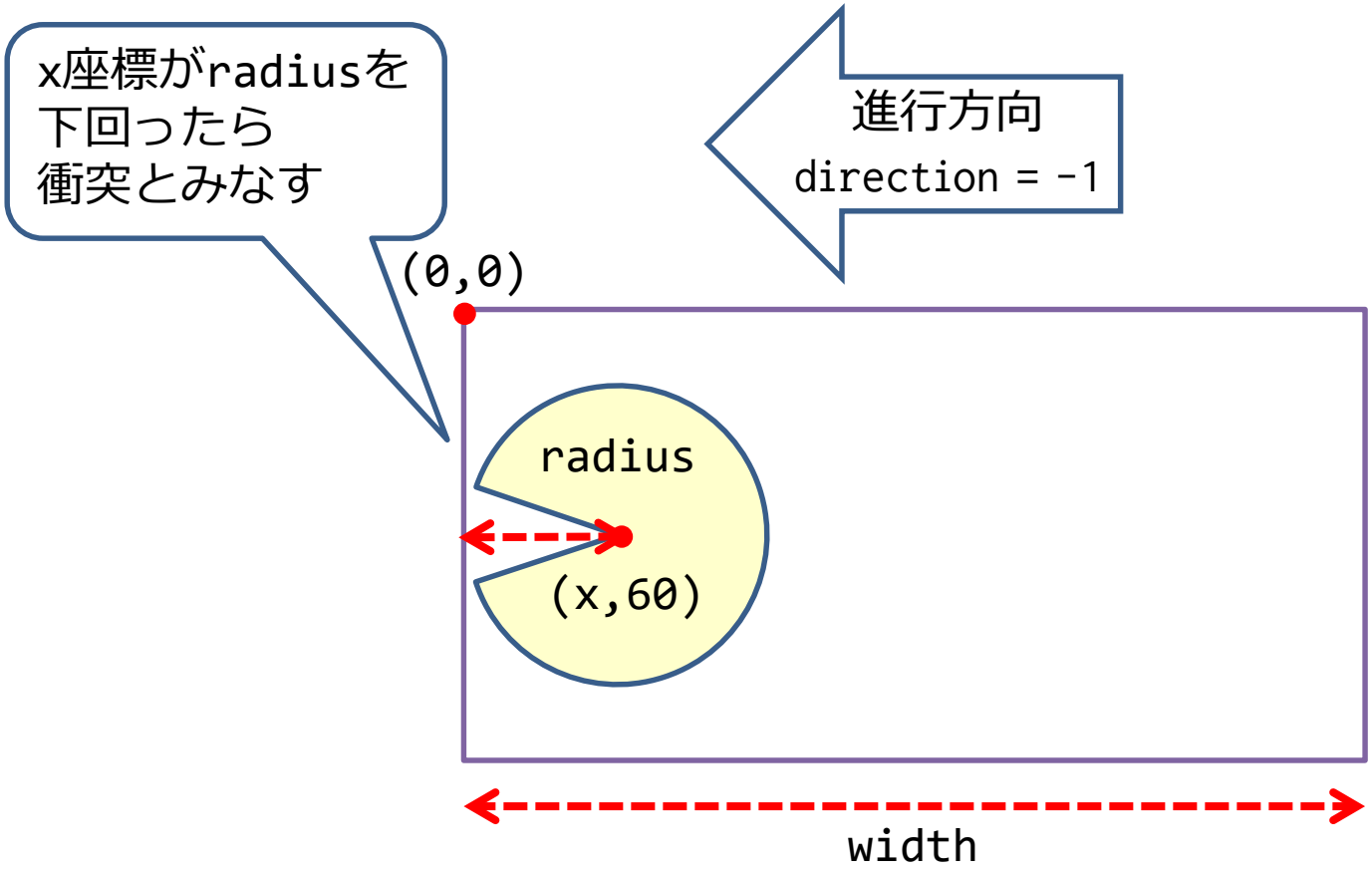

(0,0)
radius
(x,60)
width
図形のx座標は
(速度×向き)だけ加算
x += speed * direction;
directionは
右向きで+1，左向きで-1

右端で反射する場合




```
if(x > width - radius) {    // 右壁に衝突したら
  direction = -direction;   // 進行方向を逆にする
}
```

左端で反射する場合




```
if(x < radius) {              // 左壁に衝突したら
  direction = -direction;     // 進行方向を逆にする
}
```

左端・右端で反射する条件をまとめる

```
if(x > width - radius) {    // 右端に衝突したら
  direction = -direction;   // 進行方向を逆にする
}
```

```
if(x < radius) {            // 左端に衝突したら
  direction = -direction;   // 進行方向を逆にする
}
```

論理和 ||
上の2つの条件の
どちらか一方を満たす

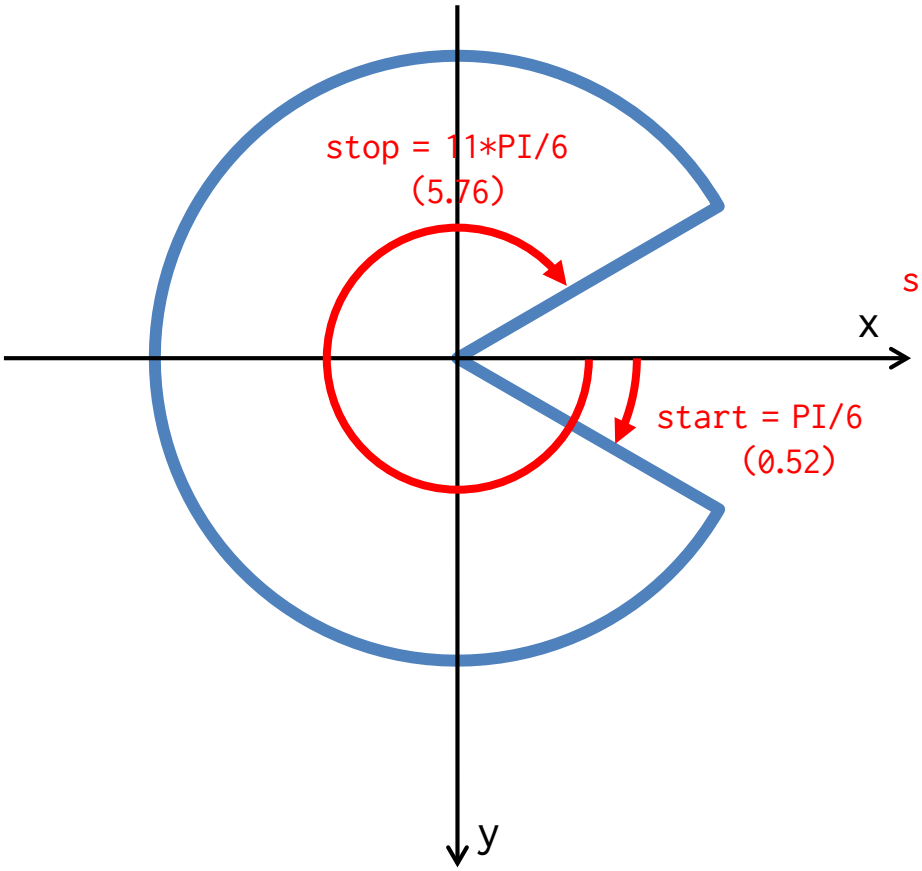
```
if((x > width - radius) || (x < radius)) { // 右端または左端に衝突したら
  direction = -direction; // 進行方向を逆にする
}
```

論理和・論理積

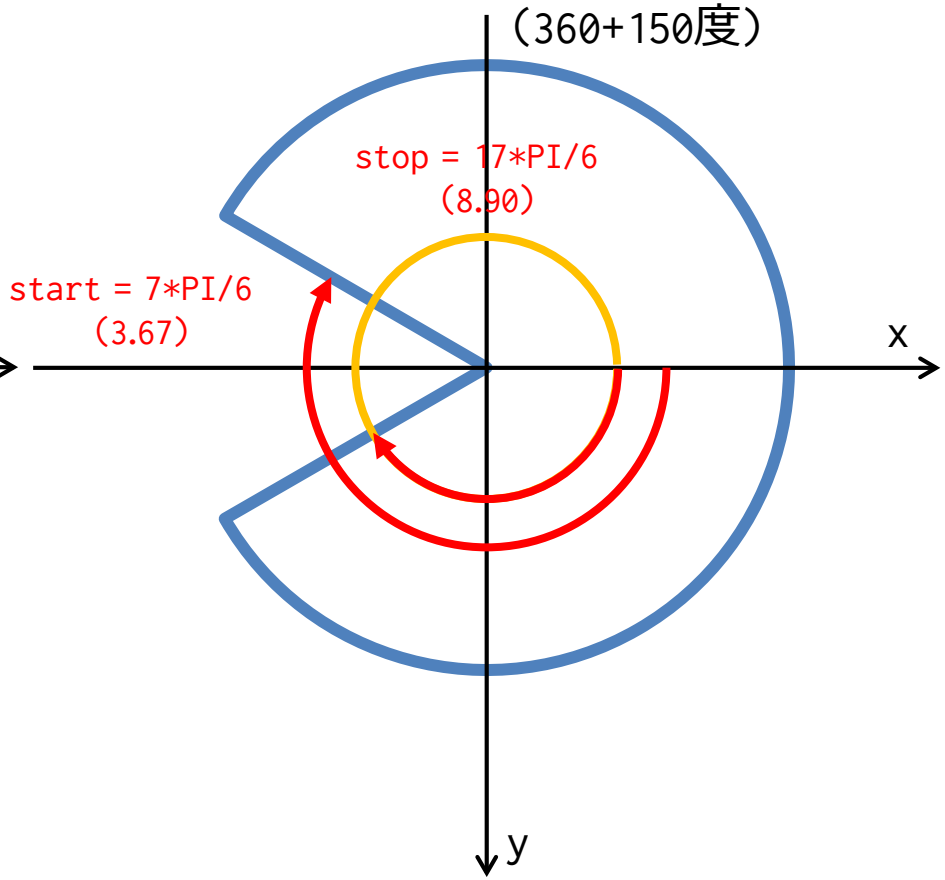
- 論理和　||
  - （条件1）||（条件2）
    条件1 または 条件2が成り立つ場合
    （どちらかの条件が真の場合）
    例）`if (mousePressed || keyPressed)`
- 論理積　&&
  - （条件1）&&（条件2）
    条件1かつ条件2が成り立つ場合(どちらも真の場合)
    例）`if (mousePressed && mouseButton == LEFT)`

開始・終了角度

右向き
開始角度 start　30度（PI/6）
終了角度 stop　330度（11*PI/6）

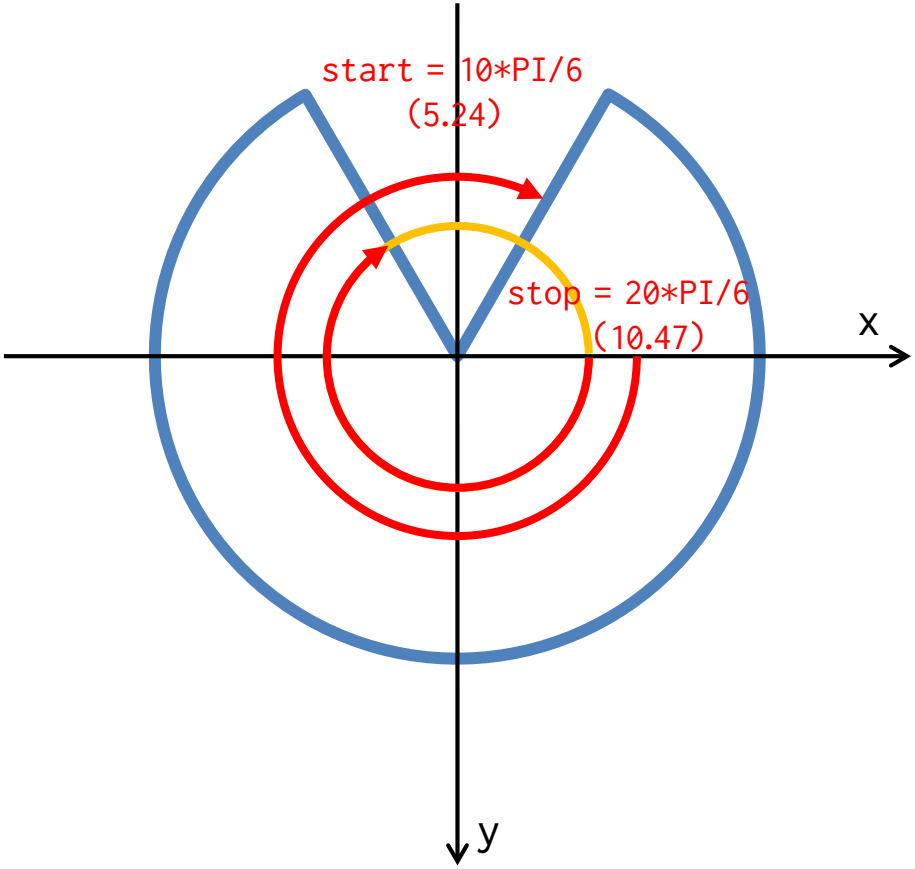


左向き
開始角度 start 210度（7*PI/6）
終了角度 stop　510度（17*PI/6）
（360+150度）

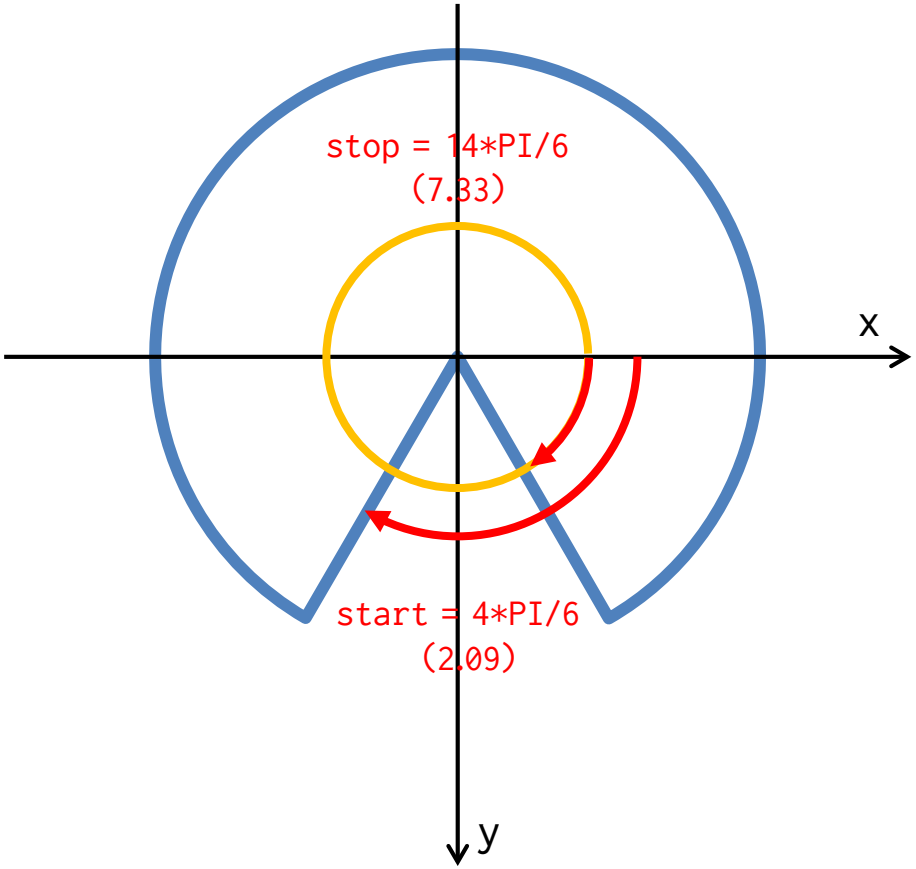


おまけ　上向き・下向き

上向き
開始角度 start 300度（10*PI/6）
終了角度 stop　600度（20*PI/6）



下向き
開始角度 start 120度（4*PI/6）
終了角度 stop　420度（14*PI/6）

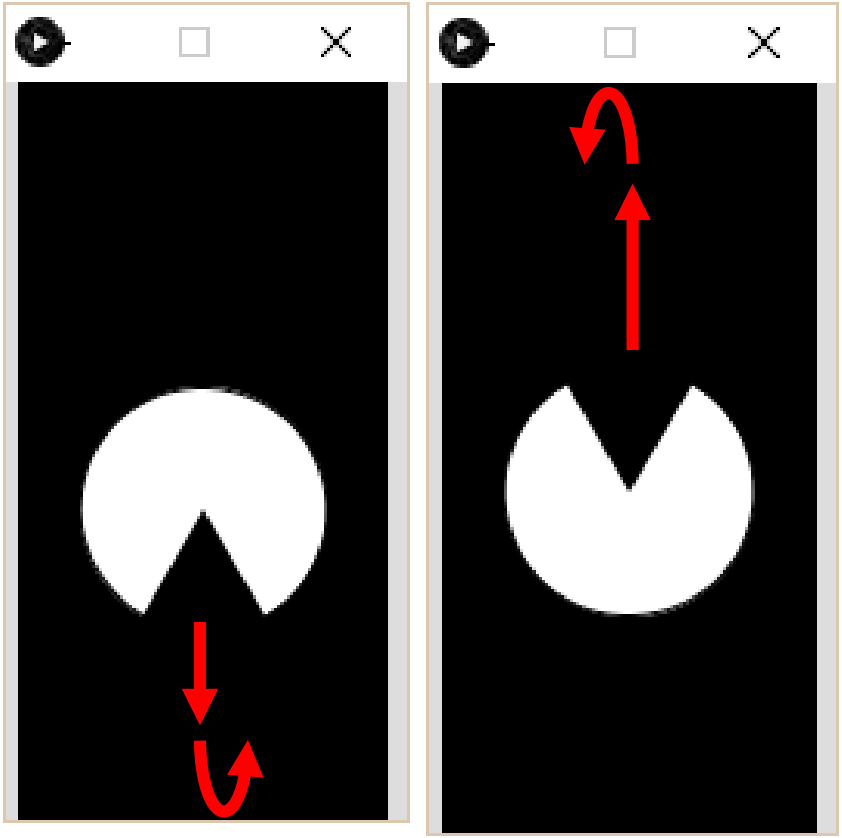


Kadai_08_03

- Example 8-5を参考に、円弧を上下で反射するプログラムを作る
  - ウィンドウサイズを(120, 240)
  - 変数xからyに変更
  - yが下端を超えたら、移動方向を反転する
  - 円弧の角度は講義資料の最後のページ下向き・上向きの場合の角度を参考にする

実行結果例

- ウィンドウサイズを
  (120, 240)
- 変数xからyに変更
- yが下端を超えたら、
  移動方向を反転する
- 下方向に進むときは、
  円弧の切れ目を下に向ける
- 上方向に進むときは、
  円弧の切れ目を上に向ける
- 円弧の角度は講義資料の最後
  のページを参考にする

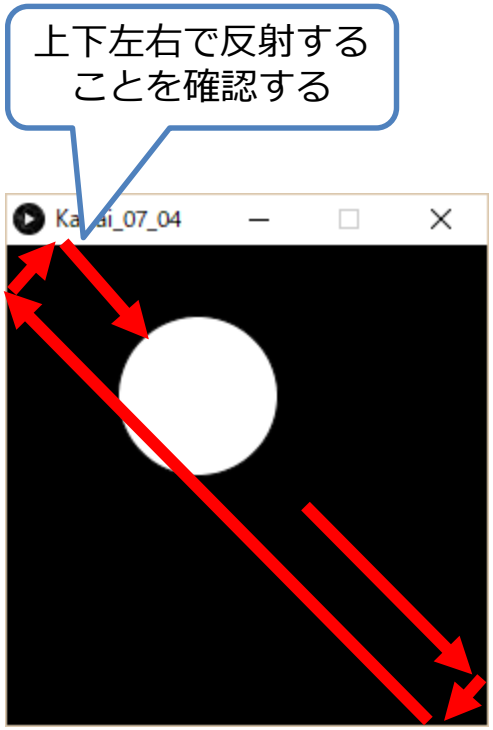


Kadai_08_04（任意提出）

- Example 8-5を参考に、円を上下左右で反射するプログラムを作る
  - 図形を円弧から円に変更
  - ウィンドウサイズを(240, 240)
  - 変数xとyを宣言、初期値はx = 90とy = 110
  - x方向の移動方向を表す変数xdirと、y方向の移動方向を表すydirを宣言、初期値はxdir = 1とydir = 1
  - xが右端または左端を超えたらx方向の移動方向を反転する、yが下端または上端を超えたらy方向の移動方向を反転する

実行結果例

- 図形を円弧から円に変更
  arc() -> ellipse()
- ウィンドウサイズを(240, 240)
- 変数xとyを宣言、
  初期値はx = 90, y = 110
- xの移動方向を表す変数xdirと、
  yの移動方向を表すydirを宣言、
  初期値はxdir = 1, ydir = 1
- xが右または左を超えたら
  xの移動方向を反転する
- yが下または上を超えたら
  yの移動方向を反転する



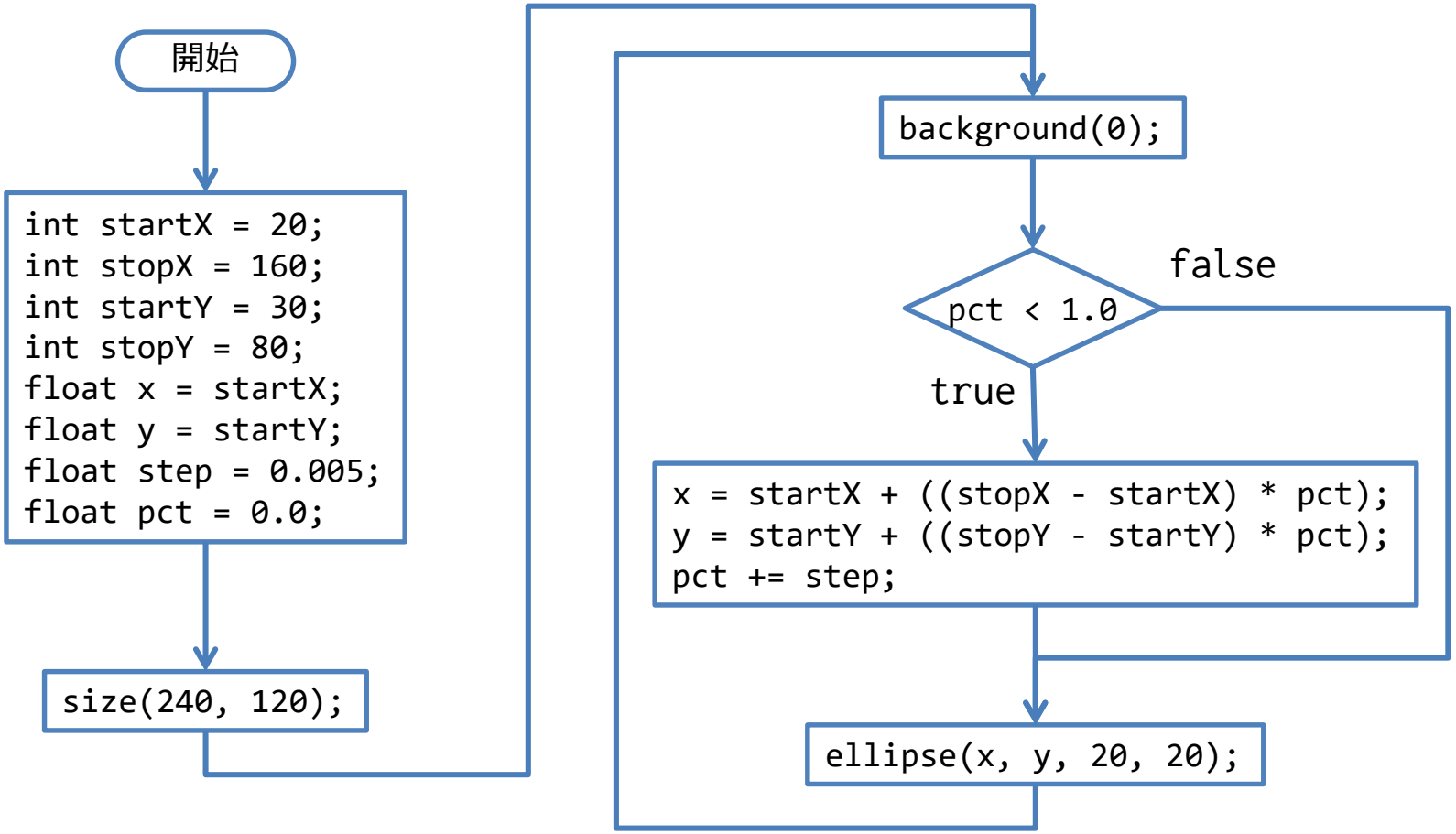
2点間の移動

- Example 8-6 2点間の経路を計算する

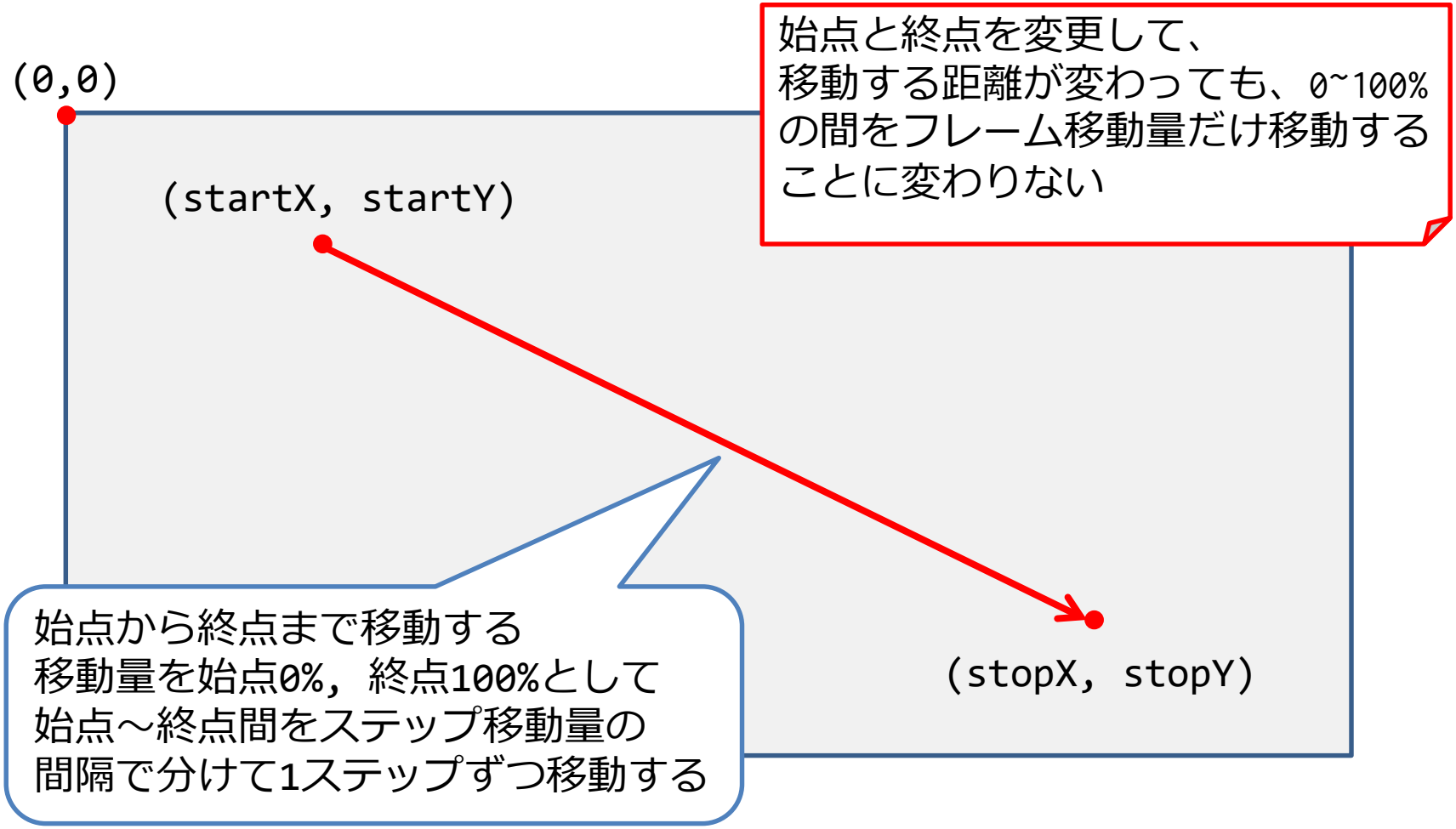
```
int startX = 20;   // 始点のx座標 20
int stopX = 160;   // 終点のx座標 160
int startY = 30;   // 始点のy座標 30
int stopY = 80;    // 終点のy座標 80
float x = startX;  // 現在のx座標 初期値 始点のx座標
float y = startY;  // 現在のy座標 初期値 始点のy座標
float step = 0.005; // 1フレームで移動する割合(0.0 ~ 1.0)
float pct = 0.0;   // 始点から終点まで移動した割合

void setup() {
  size(240, 120);
}

void draw() {
  background(0);
  if (pct < 1.0) { // 移動した割合が1.0より小さい場合
    x = startX + ((stopX - startX) * pct);     // x移動
    y = startY + ((stopY - startY) * pct);     // y移動
    pct += step;                               // 移動した割合 step加算
  }
  ellipse(x, y, 20, 20);
}
```

2点間の移動

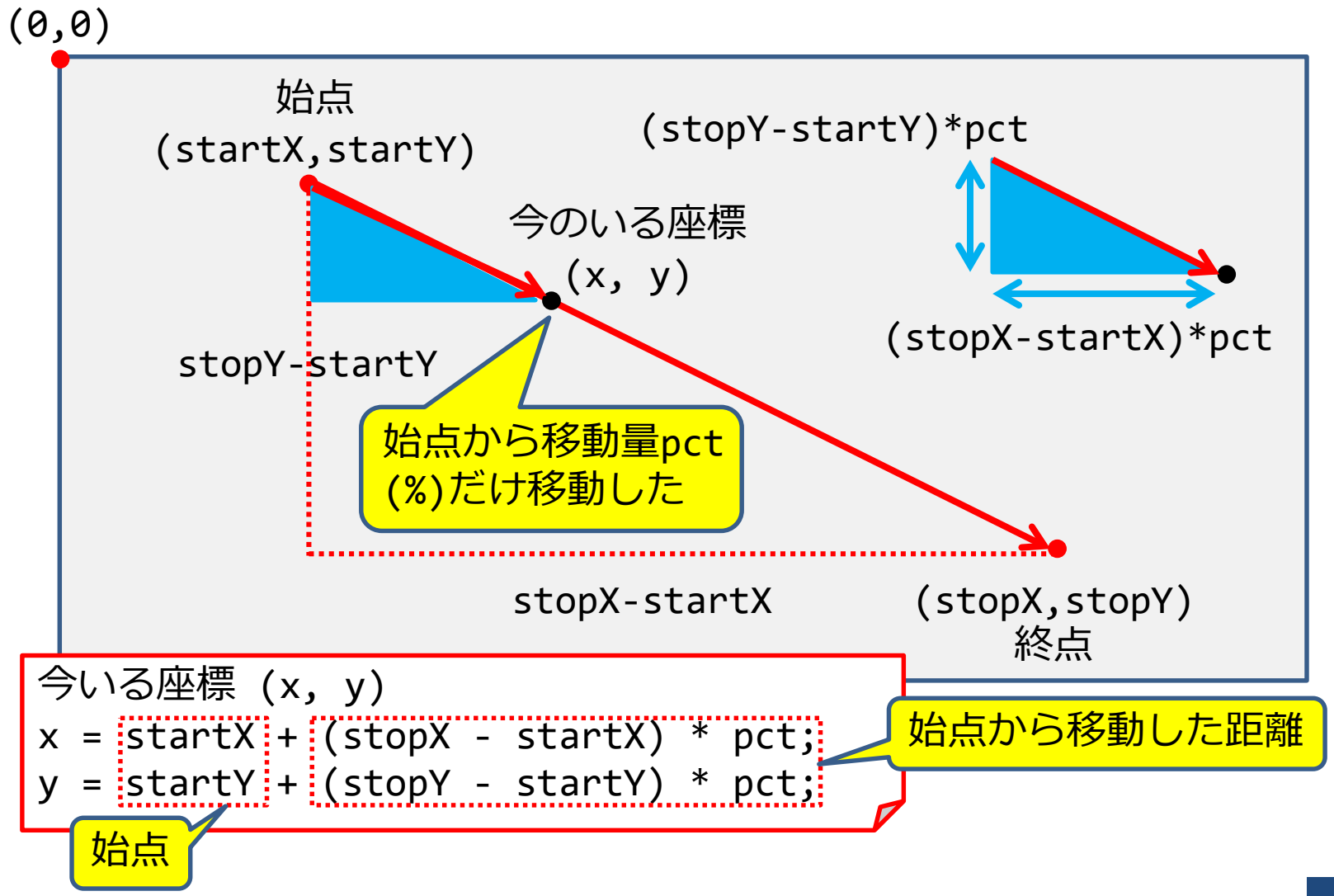
- フローチャート



2点間の移動


(0,0)
(startX, startY)
(stopX, stopY)
始点と終点を変更して、
移動する距離が変わっても、0~100%
の間をフレーム移動量だけ移動する
ことに変わりない
始点から終点まで移動する
移動量を始点0%，終点100%として
始点～終点間をステップ移動量の
間隔で分けて1ステップずつ移動する

2点間の移動距離を計算



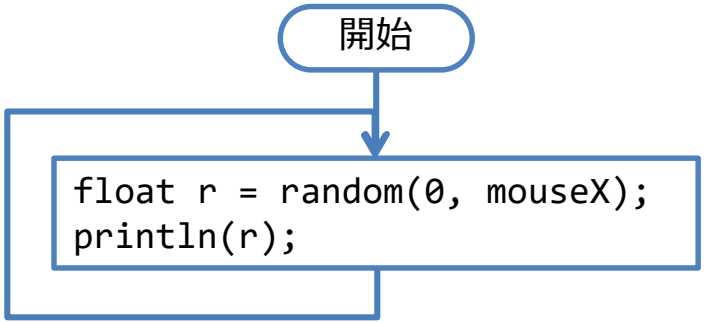
今いる座標 (x, y)

```
x = startX + (stopX - startX) * pct;
y = startY + (stopY - startY) * pct;
```

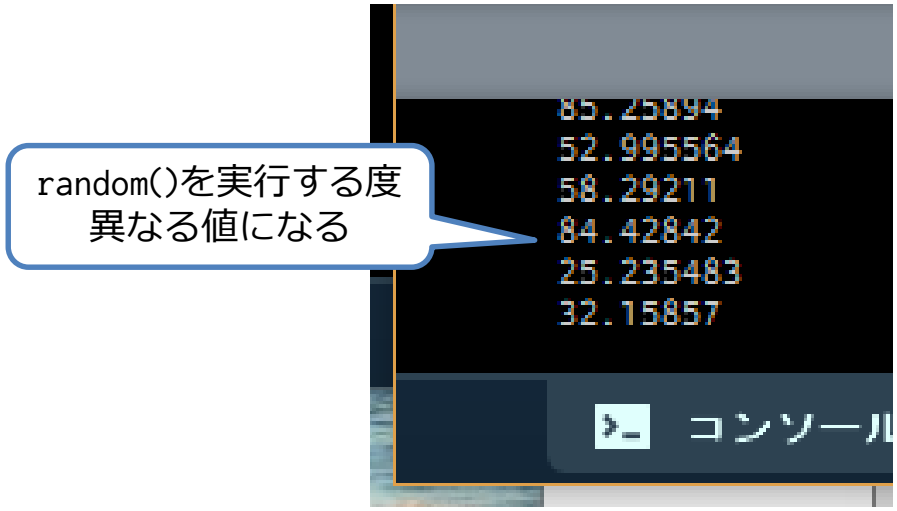
乱数

- Example 8-7 乱数の生成

```
void draw() {
  float r = random(0, mouseX);
  println(r);
}
```



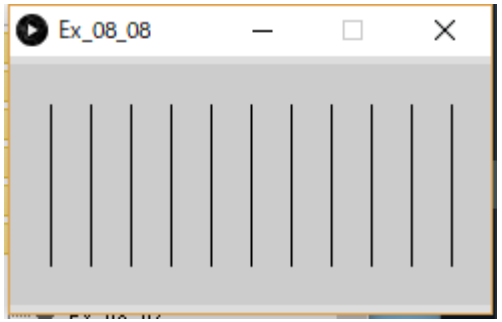
- random()関数
  - 教科書 P232 乱数の生成
    - random(high)
      0以上high未満の乱数生成
    - random(low, high)
      low以上high未満の乱数生成

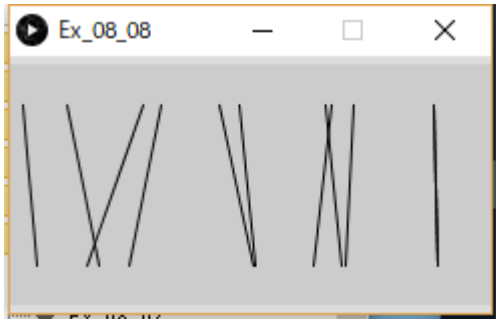


乱数

- Example 8-8 ランダムに描く

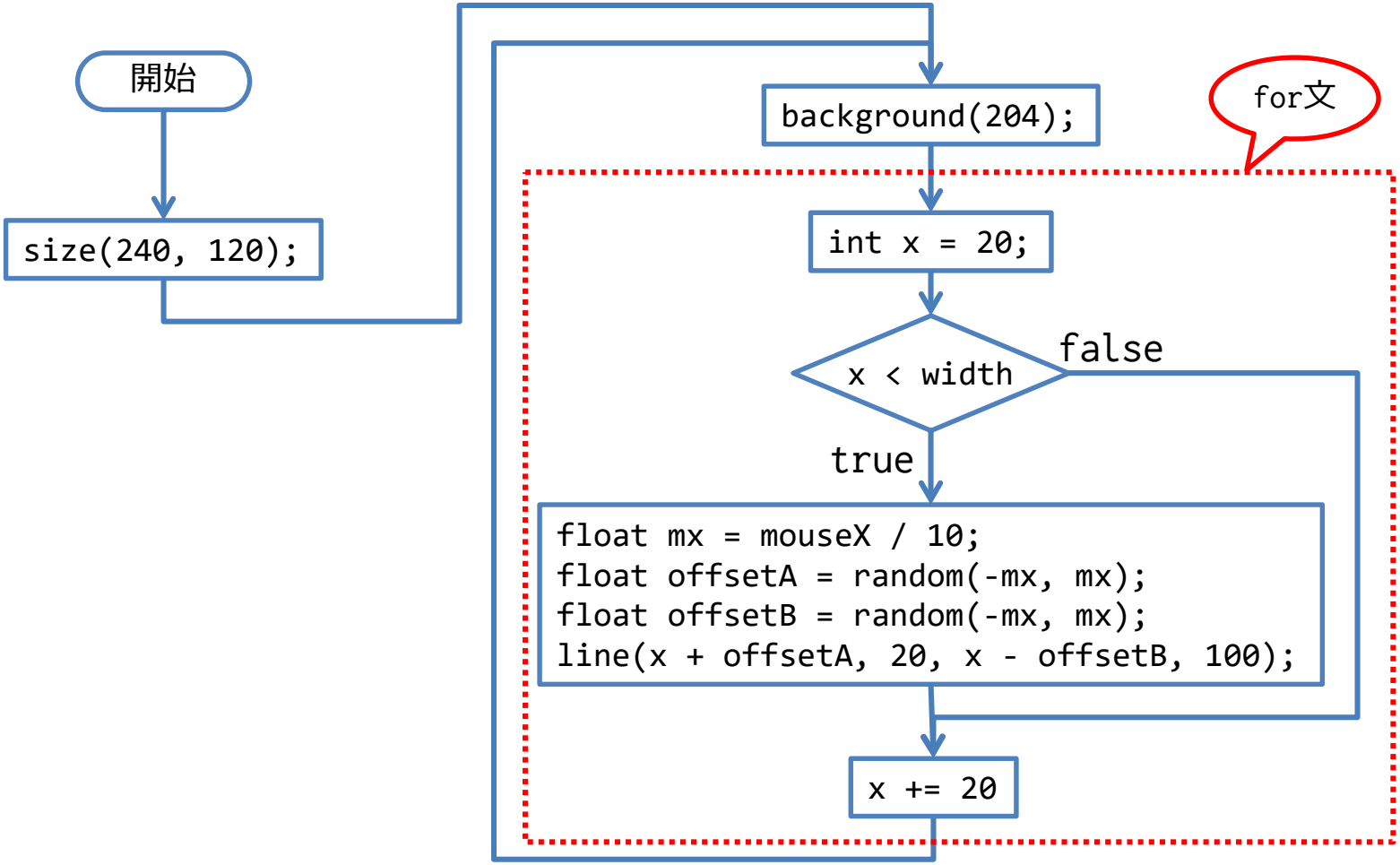
```
void setup() {
  size(240, 120);
}

void draw() {
  background(204);
  for (int x = 20; x < width; x += 20) {
    float mx = mouseX / 10;
    float offsetA = random(-mx, mx);
    float offsetB = random(-mx, mx);
    line(x + offsetA, 20, x - offsetB, 100);
  }
}
```



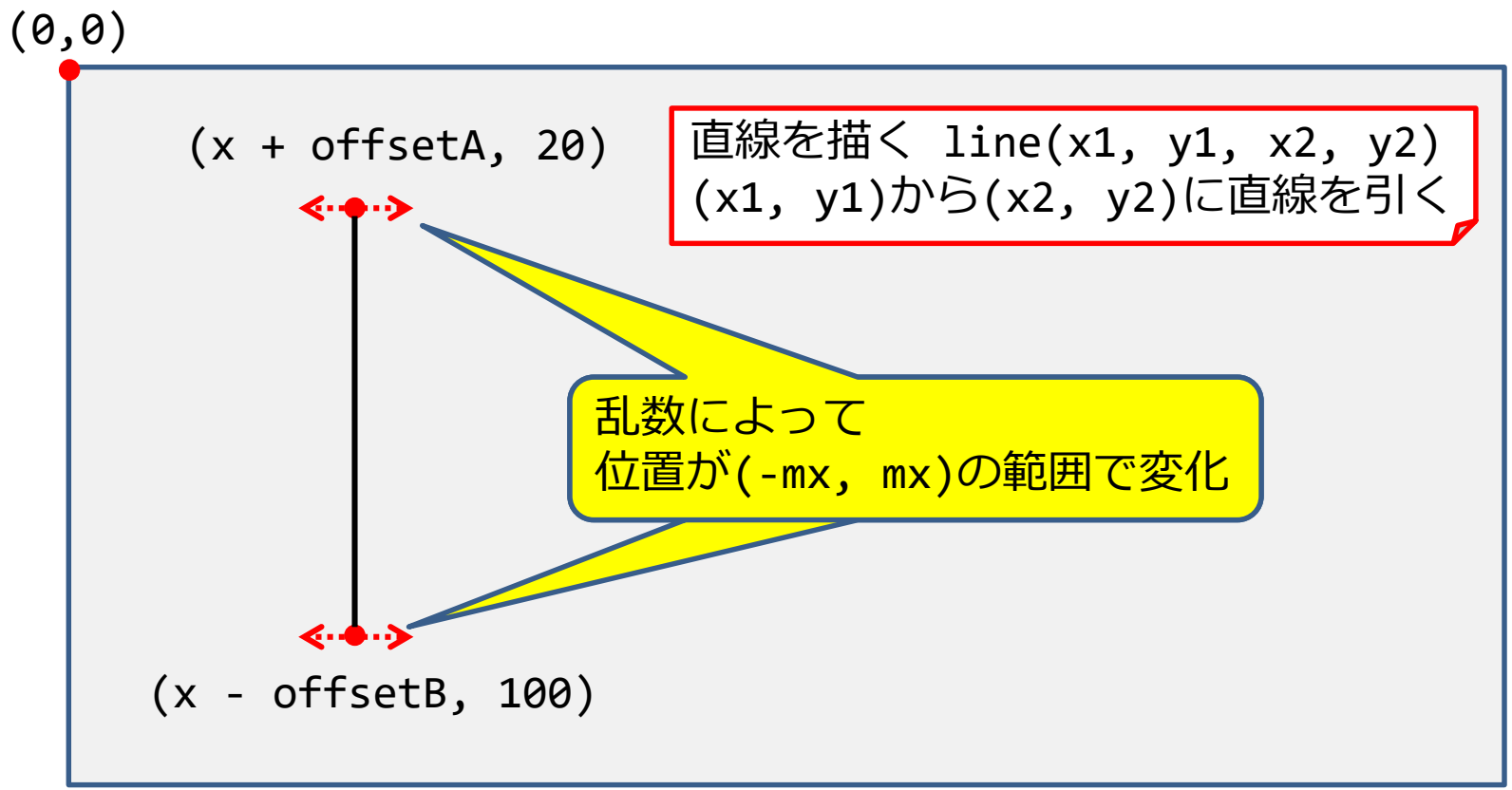


乱数

- フローチャート


開始
size(240, 120);
background(204);
for文
int x = 20;
x < width
false
true
float mx = mouseX / 10;
float offsetA = random(-mx, mx);
float offsetB = random(-mx, mx);
line(x + offsetA, 20, x - offsetB, 100);
x += 20

線

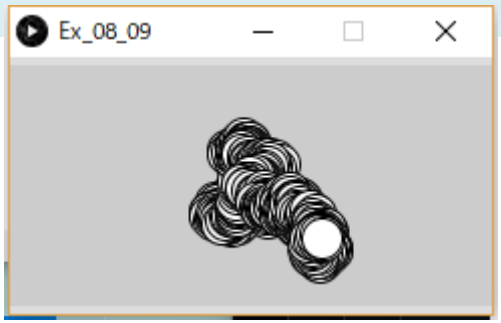

(0,0)
(x + offsetA, 20)
直線を描く line(x1, y1, x2, y2)
(x1, y1)から(x2, y2)に直線を引く
乱数によって
位置が(-mx, mx)の範囲で変化
(x - offsetB, 100)

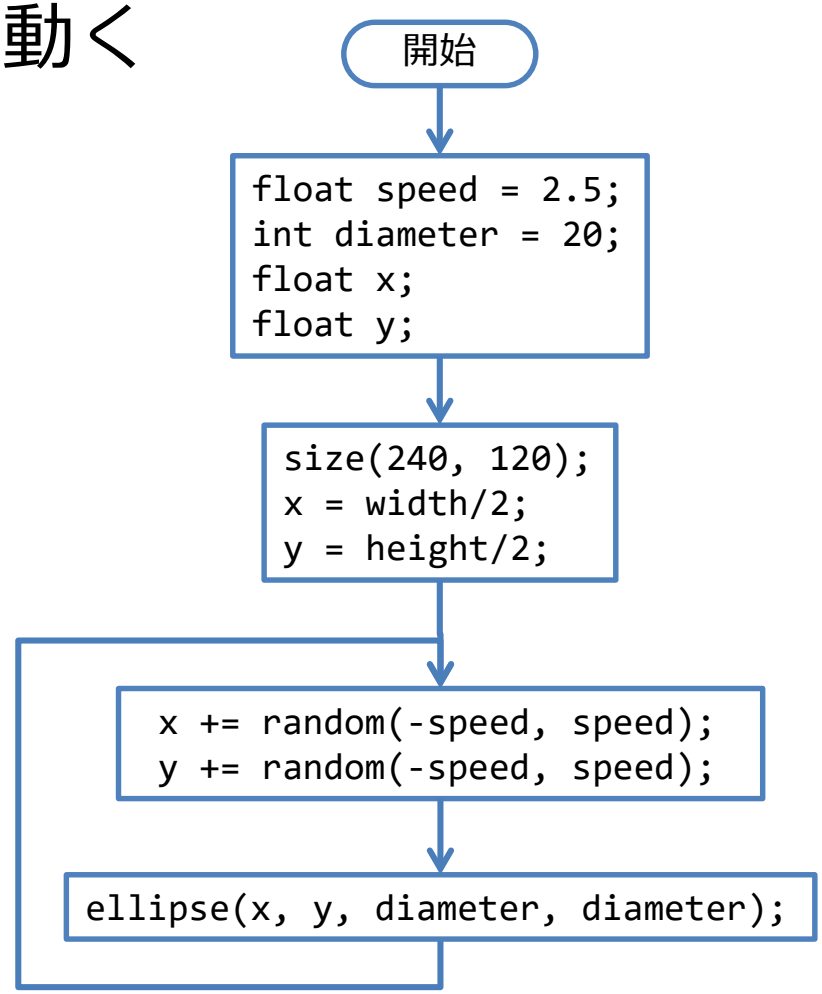
乱数

- Example 8-9 ランダムに動く

```
float speed = 2.5;
int diameter = 20;
float x;
float y;

void setup() {
  size(240, 120);
  x = width/2;
  y = height/2;
}

void draw() {
  x += random(-speed, speed);
  y += random(-speed, speed);
  ellipse(x, y, diameter, diameter);
}
```

P120
変更点



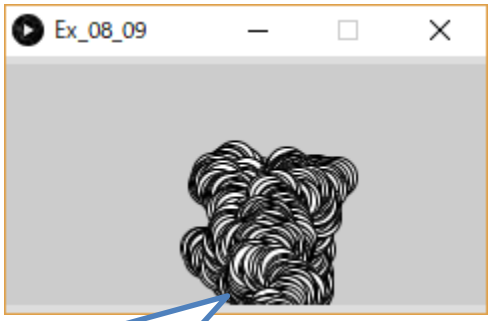


乱数

- P120 変更点
  - 円が画面外に出ないように制限
  - 値を制限する constrain() P230

    constrain(a, min, max)

    aの値をminとmaxの間に制限する



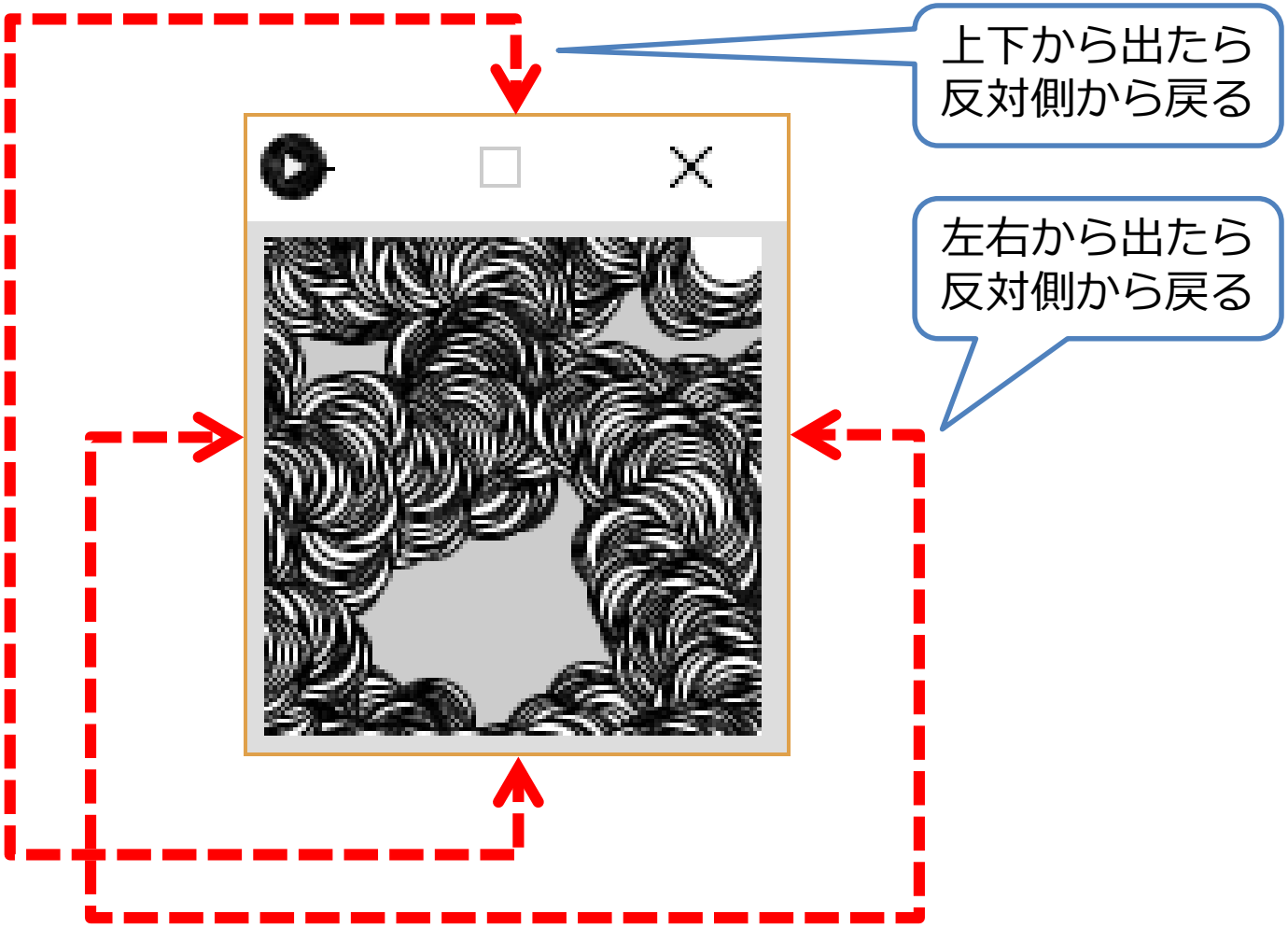
円の中心が画面外に出ない

```
void draw() {
  x += random(-speed, speed);
  y += random(-speed, speed);
  x = constrain(x, 0, width);    //xを0とwidthの間に制限
  y = constrain(y, 0, height);   //yを0とheightの間に制限
  ellipse(x, y, diameter, diameter);
}
```

Kadai_08_05

- Example 8-9 と 8-4 を参考に、ランダムに動く円が上下左右から外に出たら、反対側から入るプログラムを作成する
  - 左から出たら右から入る、右から出たら左から入る
  - 上から出たら下から入る、下から出たら上から入る
  - 注意：Example8-4は右から出て左から戻ることだけ、左から出て右から戻る・上から出て下から戻る・下から出て上から戻る動きは、自分で考えて追加する

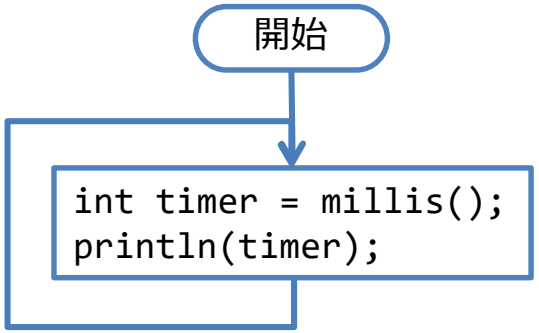
実行結果



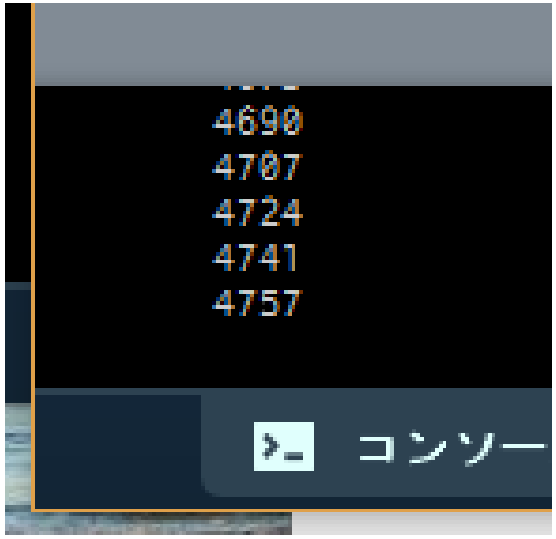
タイマー

- Example 8-10 時間の経過

```
void draw() {
  int timer = millis();
  println(timer);
}
```



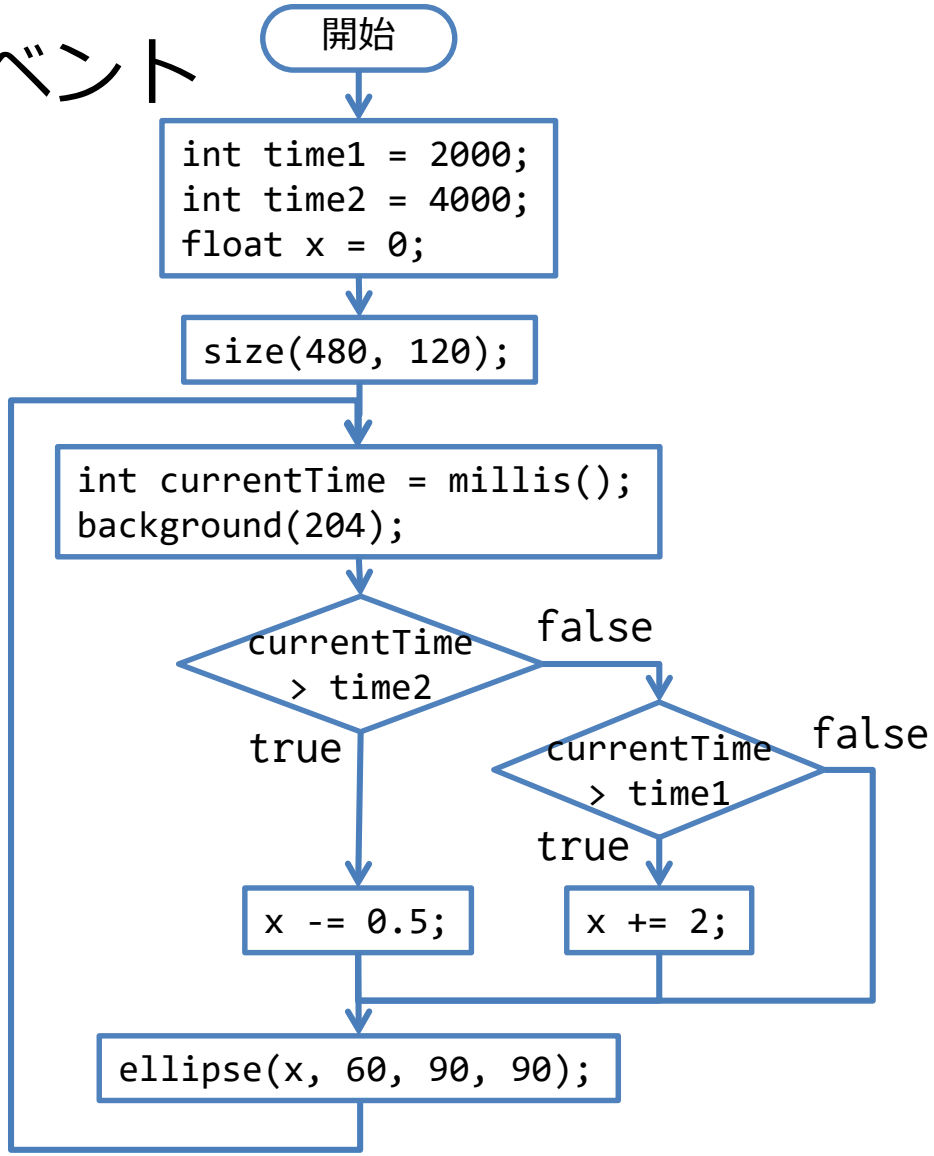
- millis()関数
  実行開始からの
  経過時間（ミリ秒）



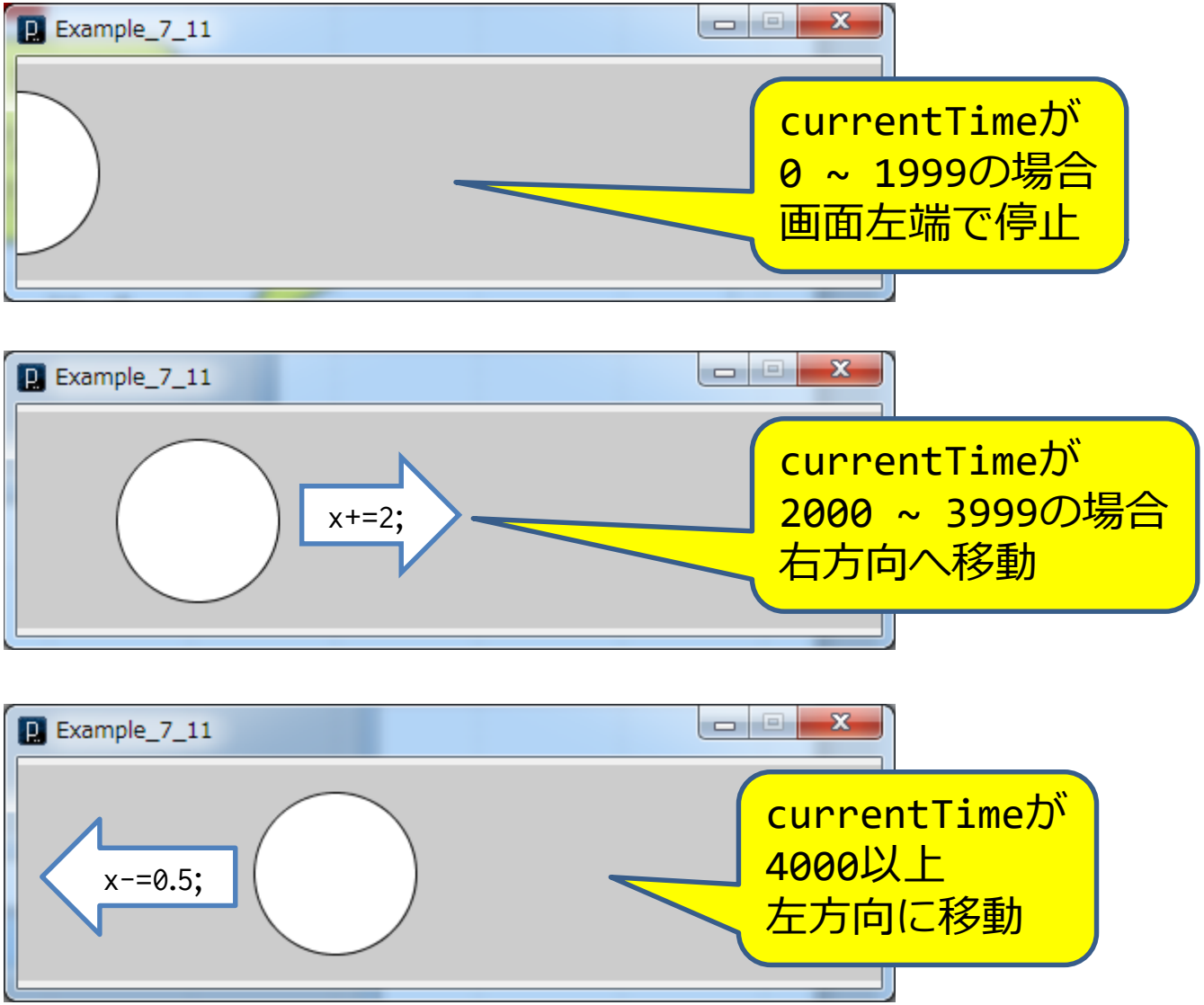
タイマー

- Example 8-11 時限式イベント

```
int time1 = 2000;
int time2 = 4000;
float x = 0;

void setup() {
  size(480, 120);
}

void draw() {
  int currentTime = millis();
  background(204);
  if (currentTime > time2) {
    x -= 0.5;
  } else if (currentTime > time1) {
    x += 2;
  }
  ellipse(x, 60, 90, 90);
}
```



実行結果


Example_7_11
currentTimeが
0 ~ 1999の場合
画面左端で停止
Example_7_11
x+=2;
currentTimeが
2000 ~ 3999の場合
右方向へ移動
Example_7_11
x-=0.5;
currentTimeが
4000以上
左方向に移動

Kadai_08_06

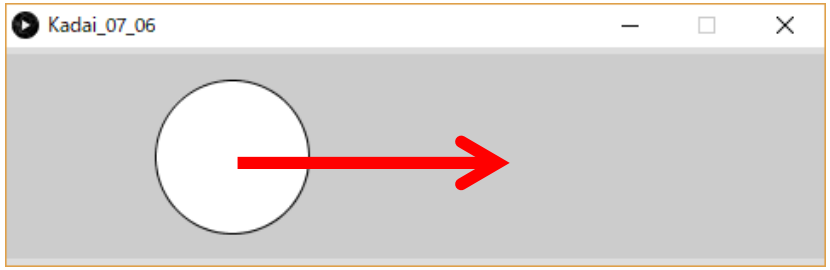
- Example 8-11 を以下のように改良
  - イベント3の設定時間を6000ミリ秒とする
  - 経過時間が6000ミリ秒を超えたら、
    x座標の値を(-1.0,1.0)の範囲のランダムな値を加算する
    (Example 8-9 を参考にするとよい)
  - 3つの条件を指定する順序をよく考える

実行結果

currentTime
0 ~ 1999

左端で停止

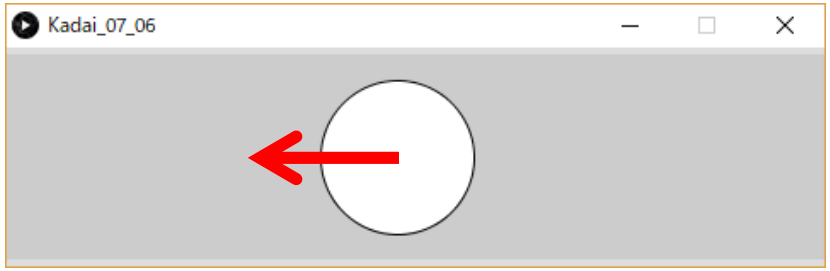
currentTime
2000 ~ 3999



右方向に移動

x += 2.0

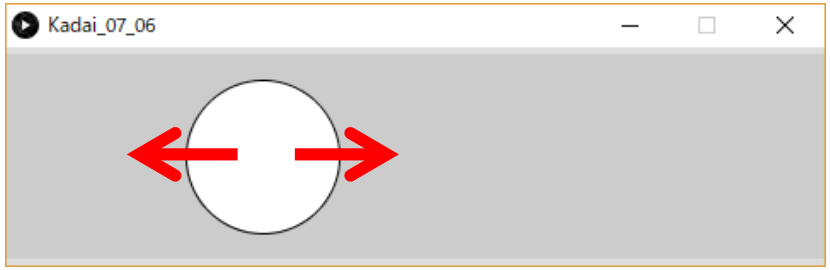
currentTime
4000 ~ 5999



左方向に移動

x -= 0.5

currentTime
6000 ~

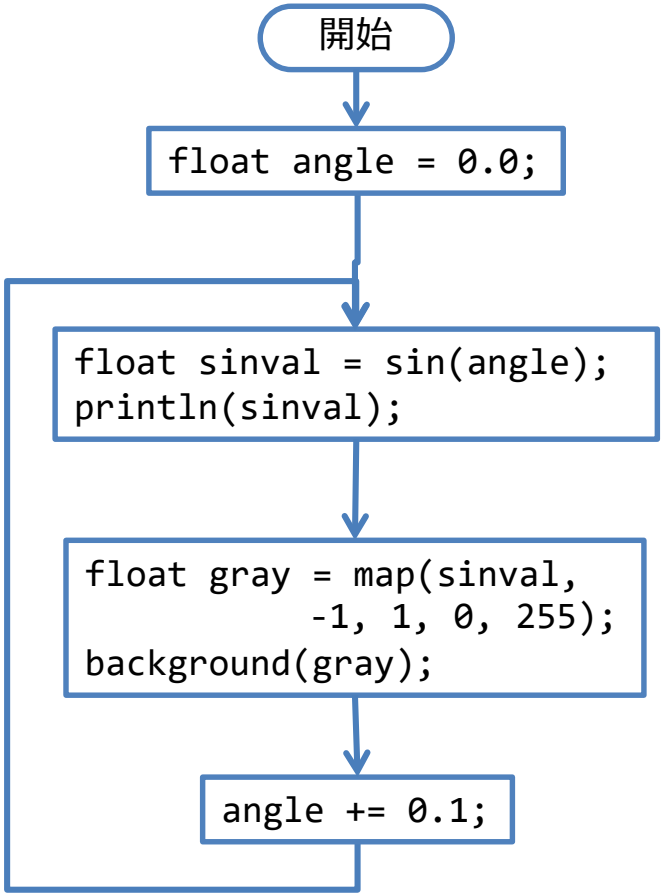


(-1.0, 1.0)の範囲で
ランダムに移動

円運動

- Example 8-12 サイン波の値

```
float angle = 0.0;

void draw() {
  float sinval = sin(angle);
  println(sinval);
  float gray = map(sinval, -1, 1, 0, 255);
  background(gray);
  angle += 0.1;
}
```

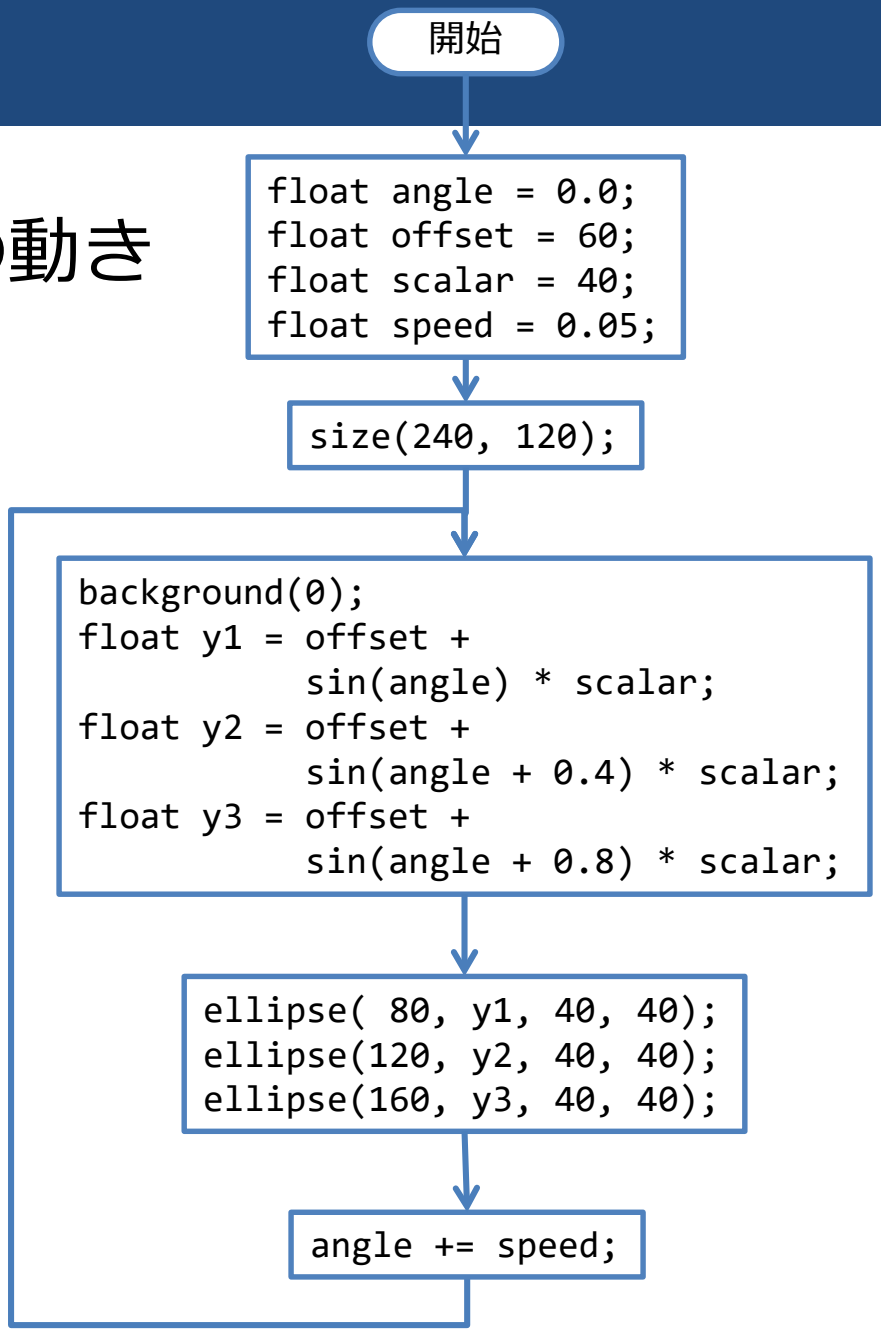


- 教科書 P231
  - sin(a)
    サイン(正弦)を求める (-1.0 ～ +1.0)

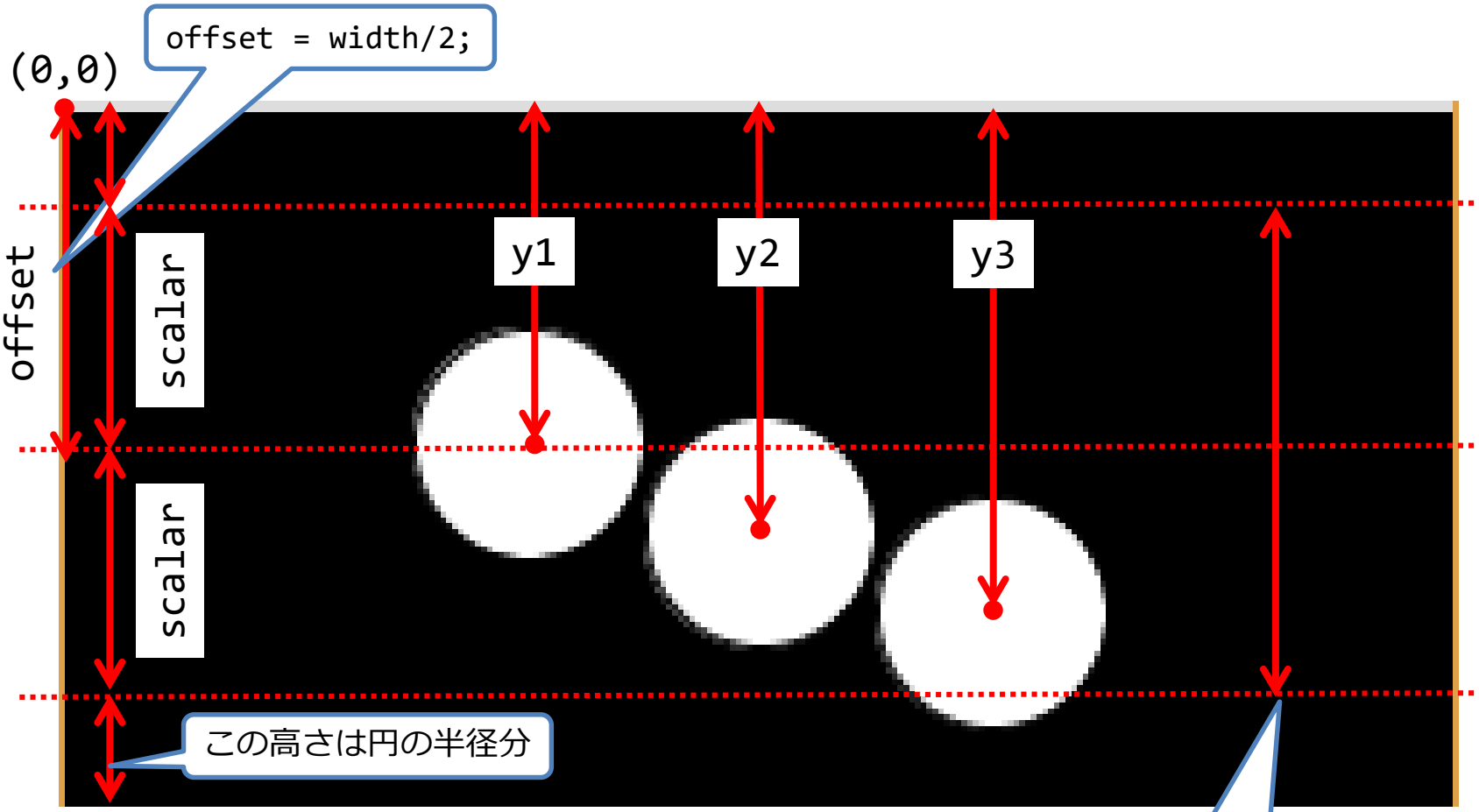
円運動

- Example 8-13 サイン波の動き

```
float angle = 0.0;
float offset = 60;
float scalar = 40;
float speed = 0.05;

void setup() {
  size(240, 120);
}

void draw() {
  background(0);
  float y1 = offset +
             sin(angle) * scalar;
  float y2 = offset +
             sin(angle + 0.4) * scalar;
  float y3 = offset +
             sin(angle + 0.8) * scalar;
  ellipse( 80, y1, 40, 40);
  ellipse(120, y2, 40, 40);
  ellipse(160, y3, 40, 40);
  angle += speed;
}
```



実行結果

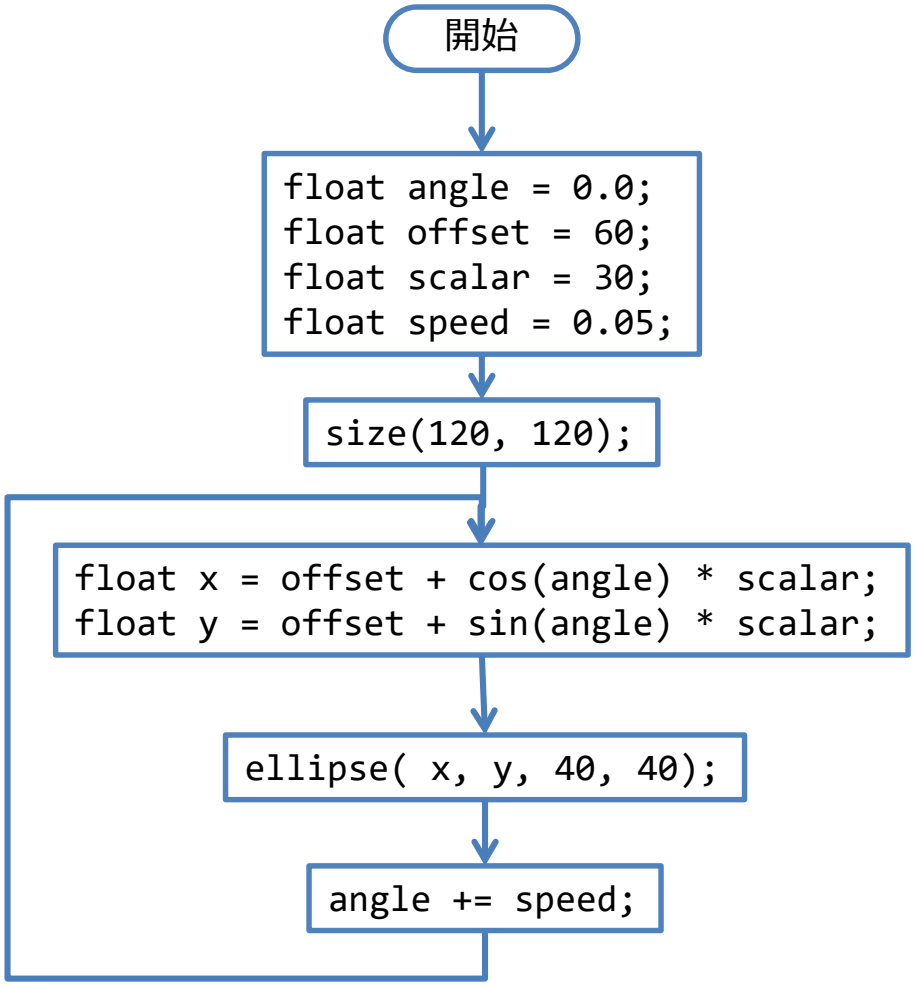



```
y1 = offset + sin(angle) * scalar;
y2 = offset + sin(angle + 0.4) * scalar;
y3 = offset + sin(angle + 0.8) * scalar;
```

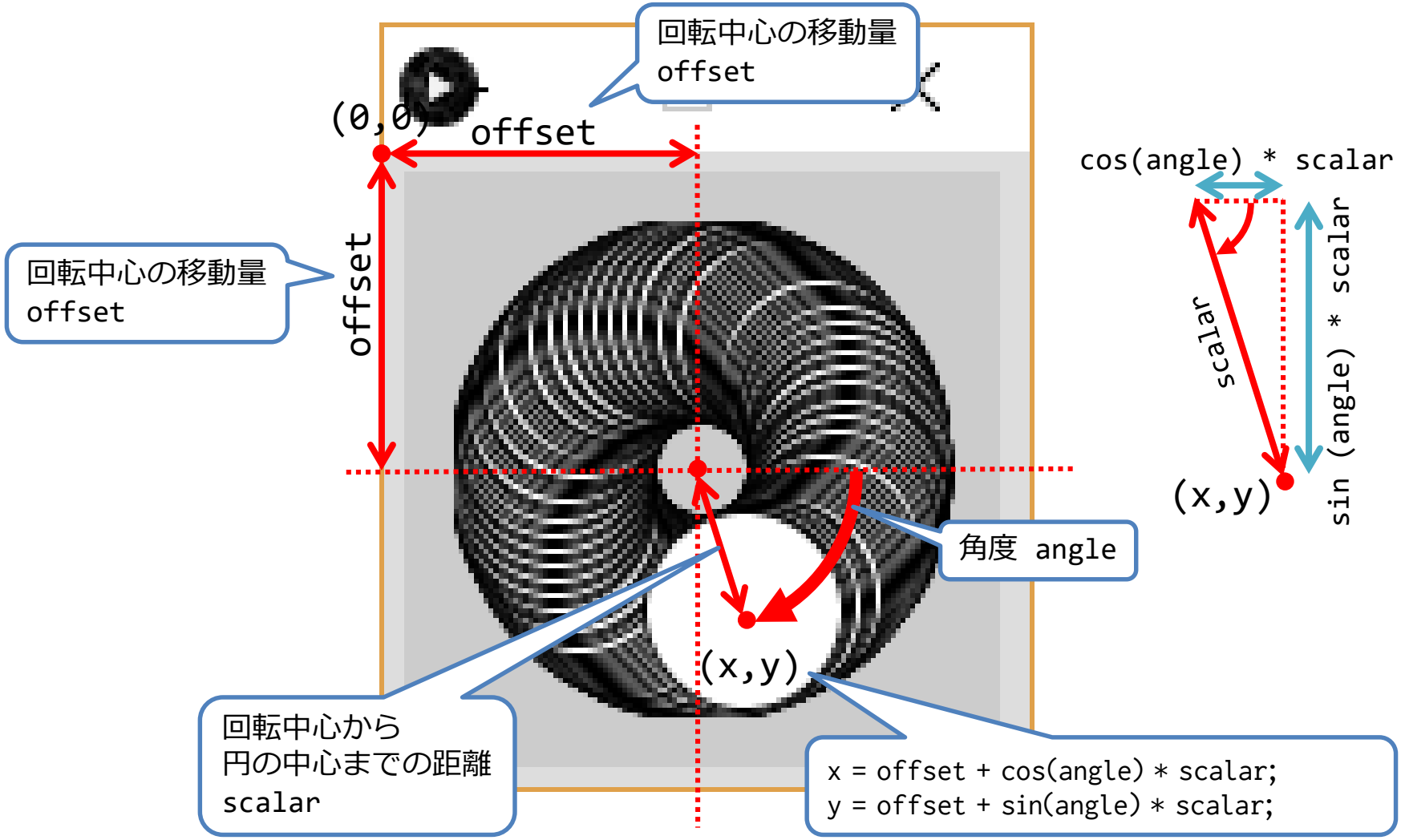
円運動

- Example 8-14 円運動

```
float angle = 0.0;
float offset = 60;
float scalar = 30;
float speed = 0.05;

void setup() {
  size(120, 120);
}

void draw() {
  float x = offset +
            cos(angle) * scalar;
  float y = offset +
            sin(angle) * scalar;
  ellipse( x, y, 40, 40);
  angle += speed;
}
```



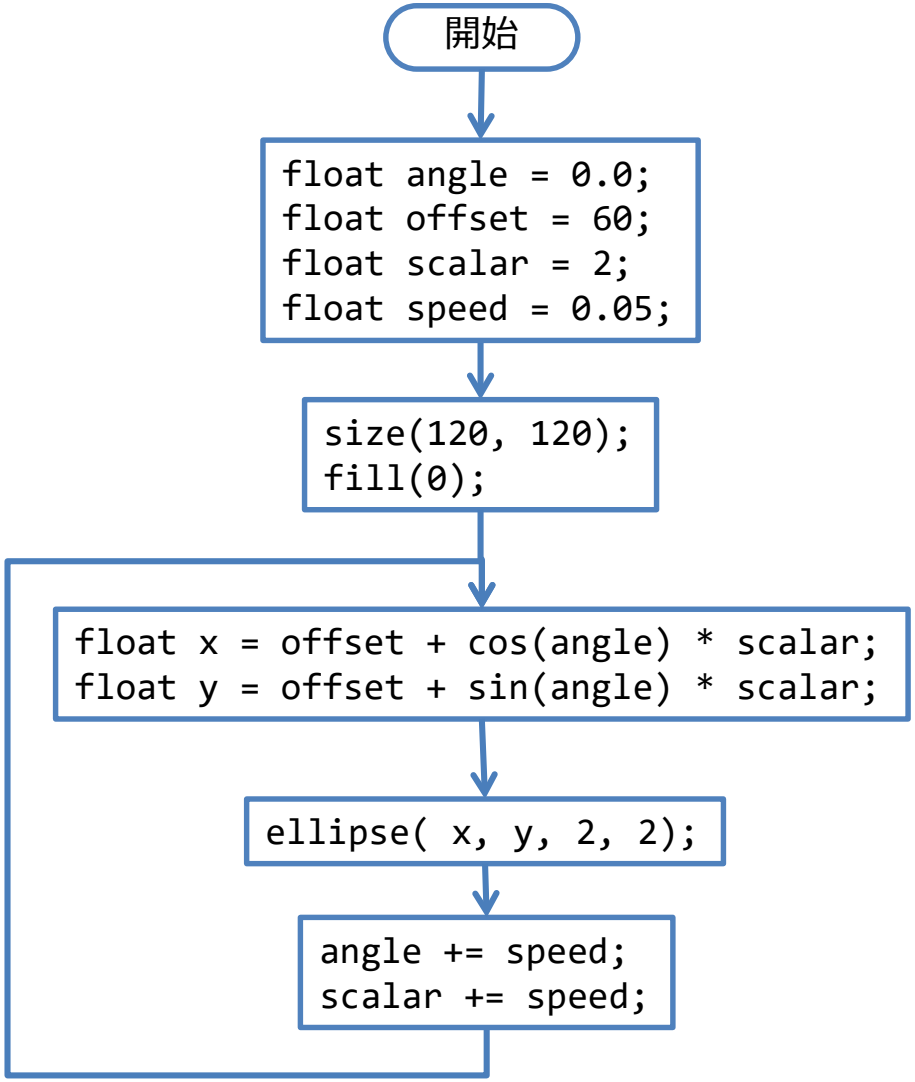
実行結果


回転中心の移動量
offset
(0,0)
offset
回転中心の移動量
offset
offset
cos(angle) * scalar
scalar
sin (angle) * scalar
(x,y)
角度 angle
(x,y)
回転中心から
円の中心までの距離
scalar
x = offset + cos(angle) * scalar;
y = offset + sin(angle) * scalar;

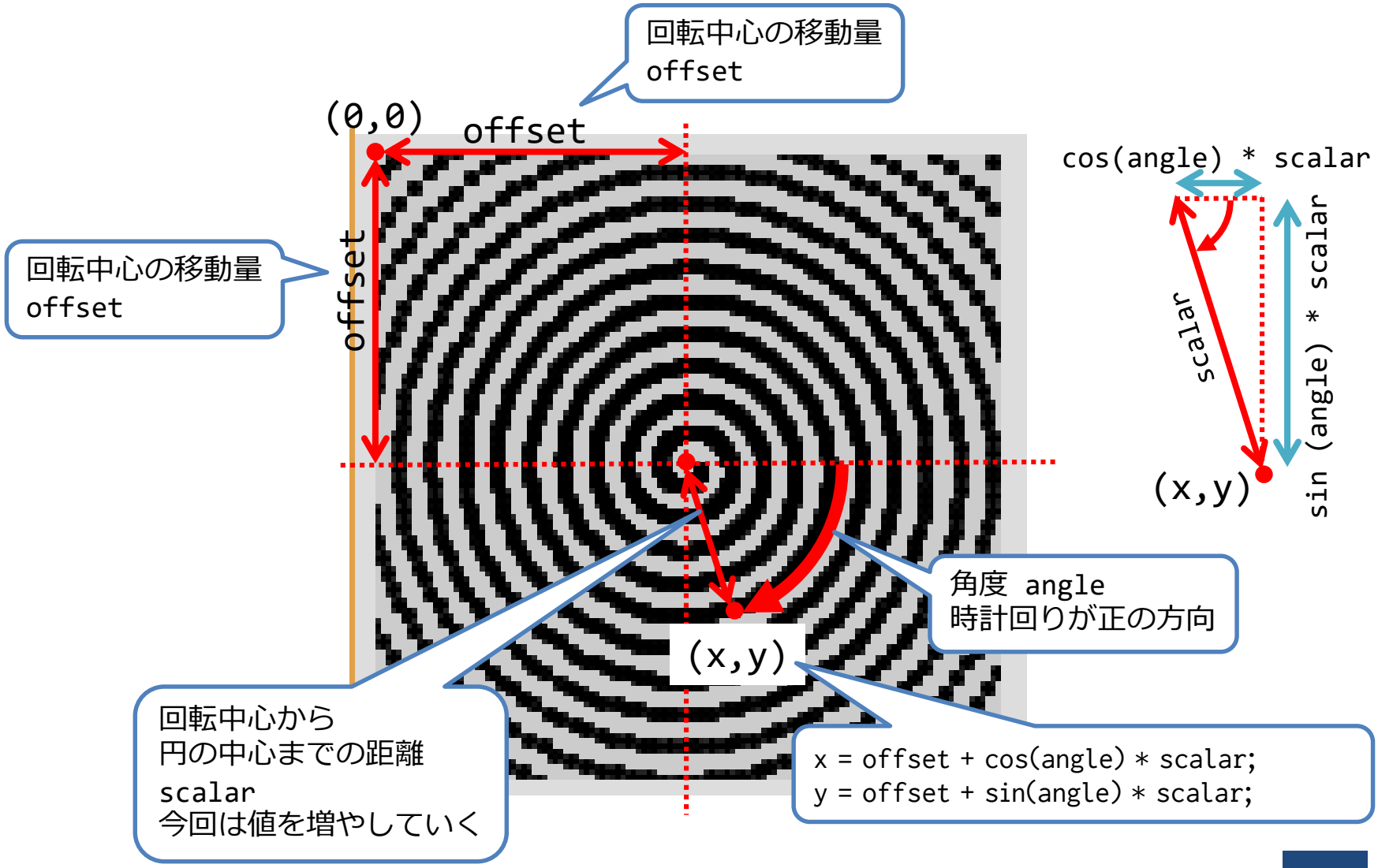
円運動

- Example 8-15 らせん

```
float angle = 0.0;
float offset = 60;
float scalar = 2;
float speed = 0.05;

void setup() {
  size(120, 120);
  fill(0);
}

void draw() {
  float x = offset +
            cos(angle) * scalar;
  float y = offset +
            sin(angle) * scalar;
  ellipse( x, y, 2, 2);
  angle += speed;
  scalar += speed;
}
```



実行結果

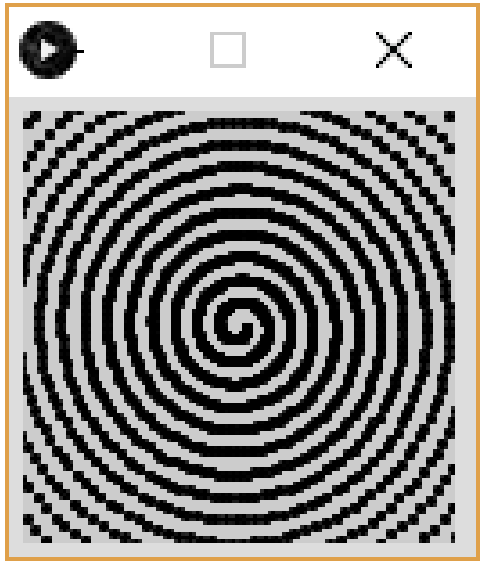
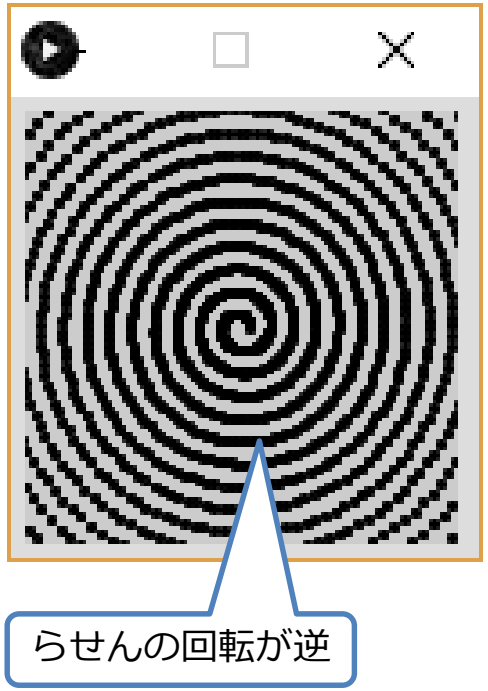

回転中心の移動量
offset
(0,0)
offset
回転中心の移動量
offset
offset
cos(angle) * scalar
scalar
sin (angle) * scalar
(x,y)
角度 angle
時計回りが正の方向
(x,y)
回転中心から
円の中心までの距離
scalar
今回は値を増やしていく
x = offset + cos(angle) * scalar;
y = offset + sin(angle) * scalar;

Kadai_08_07（任意提出）

- Example 8-15 を改良して反対回りのらせんを書くプログラムに修正する
  - Example7-15では、時計回りに回転するためangleを0.05加算する処理を行っている
  - 反時計回りに回転するためには？

実行結果

Example 7-15
実行結果



Kadai_08_07
実行結果